CS Monitor

Codice completo del progetto - generato il 14/10/2026 15:33
File: 49 | Righe di testo: 4722 | File omessi (binari/grandi): 2
Esclusi: .gitignore, cartelle di sistema/cache e file con segreti (.env, chiavi).

Struttura del progetto

```
CS Monitor/
├── .gitignore
├── app.py
├── README.md
├── requirements.txt
├── test_account.py
├── test_account_database.py
├── test_correzioni.py
├── test_database.py
├── test_importazione_log.py
├── test_modelli.py
├── test_repository.py
├── test_support.py
├── database/
│   ├── categorie_repository.py
│   ├── connessione.py
│   ├── eventi_repository.py
│   ├── importazioni_repository.py
│   ├── migrazioni.py
│   ├── strumenti.py
│   └── utenti_repository.py
├── esempi/
│   └── README.md
├── models/
│   ├── categoria.py
│   └── evento_sicurezza.py
├── routes/
│   └── account.py
├── scripts/
│   └── setup_account.py
├── services/
│   ├── account.py
│   └── log_importer.py
├── static/
│   ├── css/
│   │   └── stile.css
│   ├── img/
│   │   ├── logo.png   [omesso: binario]
│   │   └── profilo.png   [omesso: binario]
│   └── js/
│       └── importa_log.js
└── templates/
    ├── 404.html
    ├── amministrazione.html
    ├── base_account.html
    ├── dettagli_evento.html
    ├── dettagli_importazione.html
    ├── errore_account.html
    ├── errore_database.html
    ├── eventi.html
    ├── footer.html
    ├── home.html
    ├── importa_log.html
    ├── importazioni.html
    ├── info.html
    ├── login.html
    ├── modifica_evento.html
    ├── navbar.html
    ├── nuovo_evento.html
    ├── profilo.html
    └── registrazione.html
```

Indice dei file

.gitignore 3

app.py 3
README.md 8
requirements.txt 15
test_account.py 15
test_account_database.py 17
test_correzioni.py 21
test_database.py 23
test_importazione_log.py 23
test_modelli.py 26
test_repository.py 27
test_support.py 28
database/categorie_repository.py 28
database/connessione.py 29
database/eventi_repository.py 30
database/importazioni_repository.py 34
database/migrazioni.py 35
database/strumenti.py 36
database/utenti_repository.py 36
esempi/README.md 38
models/categoria.py 39
models/evento_sicurezza.py 39
routes/account.py 41
scripts/setup_account.py 43
services/account.py 43
services/log_importer.py 45
static/css/stile.css 46
static/img/logo.png [omesso] 50
static/img/profilo.png [omesso] 50
static/js/importa_log.js 50
templates/404.html 51
templates/amministrazione.html 51
templates/base_account.html 53
templates/dettagli_evento.html 54
templates/dettagli_importazione.html 55
templates/errore_account.html 56
templates/errore_database.html 56
templates/eventi.html 56
templates/footer.html 59
templates/home.html 59
templates/importa_log.html 60
templates/importazioni.html 62
templates/info.html 63
templates/login.html 65
templates/modifica_evento.html 65
templates/navbar.html 66
templates/nuovo_evento.html 67
templates/profilo.html 68
templates/registrazione.html 69

.gitignore

8 righe

```
.venv/
__pycache__/
*.pyc
.env
*.log
!esempi/*.log
.pytest_cache/
instance/credenziali-admin.txt
```

app.py

380 righe

```
from flask import Flask, render_template, abort, request, redirect, url_for, send_file, flash, g
from database.eventi_repository import (
    visualizzaEventi,
    cercaEvento,
    inserisciEvento,
    modificaEvento,
    eliminaEvento,
    statisticheEventi,
)
from models.evento_sicurezza import EventoSicurezza
from database.categorie_repository import visualizzaCategorie
import mysql.connector
import os
import csv
import secrets
import base64
from werkzeug.utils import secure_filename
from services.account import configuraSicurezza
from routes.account import account
from database.importazioni_repository import (
    salvaImportazione,
    visualizzaImportazioni,
    cercaImportazione,
)
from io import BytesIO, StringIO
from datetime import datetime
from itsdangerous import URLSafeTimedSerializer, BadSignature
from services.log_importer import analizzaLog, MAX_BYTES

app = Flask(__name__)
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY") or secrets.token_hex(32)
app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024
app.config["MAX_FORM_MEMORY_SIZE"] = 2 * 1024 * 1024
app.config["SESSION_COOKIE_SECURE"] = os.getenv("SESSION_COOKIE_SECURE", "false").lower() == "true"
app.register_blueprint(account)
configuraSicurezza(app)


def ambitoEventi():
    return dict(id_utente=g.utente['id_utente'], amministratore=g.utente['ruolo'] == 'admin')


@app.errorhandler(400)
@app.errorhandler(403)
def richiestaNonConsentita(errore):
    return (
        render_template('errore_account.html', messaggio=errore.description, codice=errore.code),
        errore.code,
    )


@app.get('/importazioni')
def storicoImportazioni():
    return render_template(
        'importazioni.html', importazioni=visualizzaImportazioni(**ambitoEventi())
    )


@app.get('/importazioni/<int:id_importazione>')
def dettagliImportazione(id_importazione):
    importazione = cercaImportazione(id_importazione, **ambitoEventi())
```

```
    if importazione is None:
        abort(404)
    eventi = visualizzaEventi(id_importazione=id_importazione, **ambitoEventi())
    return render_template('dettagli_importazione.html', importazione=importazione, eventi=eventi)


@app.get('/importazioni/<int:id_importazione>/file')
def scaricaLog(id_importazione):
    importazione = cercaImportazione(id_importazione, **ambitoEventi())
    if importazione is None:
        abort(404)
    return send_file(
        BytesIO(importazione['contenuto']),
        mimetype='text/plain',
        as_attachment=True,
        download_name=importazione['nome_file'],
    )


@app.errorhandler(mysql.connector.Error)
def erroreDatabase(errore):
    app.logger.error("Operazione sul database non riuscita.")
    return render_template("errore_database.html"), 503


@app.errorhandler(413)
def richiestaTroppoGrande(errore):
    return (
        render_template(
            "importa_log.html",
            errore="Caricamento troppo grande. Il file deve essere al massimo di 256 KB.",
        ),
        413,
    )


@app.route("/eventi/importa", methods=["GET", "POST"])
def importaLog():
    eventi, errori, categorie = [], [], []
    errore = messaggio = token = None
    firma = URLSafeTimedSerializer(app.config["SECRET_KEY"], salt="importazione-log")
    if request.method == "POST":
        try:
            if request.form.get("azione") == "conferma":
                token = request.form.get("anteprima", "")
                try:
                    anteprima = firma.loads(token, max_age=1800)
                    campi = {
                        'id_utente',
                        'eventi',
                        'nome_file',
                        'contenuto',
                        'righe_errate',
                        'identificativo',
                    }
                    if not isinstance(anteprima, dict) or not campi.issubset(anteprima):
                        raise ValueError('Anteprima non valida. Carica nuovamente il file.')
                    if anteprima['id_utente'] != g.utente['id_utente']:
                        raise ValueError('Questa anteprima appartiene a un altro account.')
                    eventi = anteprima['eventi']
                except BadSignature:
                    token = None
                    raise ValueError(
                        "Anteprima scaduta o non valida. Carica nuovamente il file."
                    ) from None
                categorie = visualizzaCategorie()
                selezionati = set(request.form.getlist("selezionati"))
                disponibili = {str(e["riga"]) for e in eventi}
                if not selezionati or not selezionati.issubset(disponibili):
                    raise ValueError("Seleziona almeno un evento valido da importare.")
                categorie_valide = {c["id_categoria"] for c in categorie}
                da_salvare = []
                for e in eventi:
                    if str(e["riga"]) not in selezionati:
```

```
                    continue
                id_categoria = request.form.get(f"categoria_{e['riga']}", type=int)
                if id_categoria not in categorie_valide:
                    raise ValueError(f"Seleziona una categoria valida per la riga {e['riga']}.")
                da_salvare.append(
                    EventoSicurezza(
                        None,
                        id_categoria,
                        e["indirizzo_ip"],
                        e["descrizione"],
                        e["gravita"],
                        data_evento=datetime.strptime(e["data_evento"], "%Y-%m-%d %H:%M:%S"),
                    )
                )
            id_importazione = salvaImportazione(
                da_salvare,
                g.utente['id_utente'],
                anteprima['nome_file'],
                base64.b64decode(anteprima['contenuto']),
                len(eventi),
                anteprima['righe_errate'],
                anteprima['identificativo'],
            )
            flash(
                f"Importazione completata: {len(da_salvare)} eventi salvati con stato Nuovo.",
                'success',
            )
            return redirect(
                url_for('dettagliImportazione', id_importazione=id_importazione), code=303
            )
        else:
            file = request.files.get("file_log")
            if file is None or not file.filename:
                raise ValueError("Seleziona un file da caricare.")
            if os.path.splitext(file.filename)[1].lower() not in (".log", ".txt"):
                raise ValueError("Sono ammessi soltanto file .log e .txt.")
            contenuto = file.read(MAX_BYTES + 1)
            eventi, errori = analizzaLog(contenuto)
            if eventi:
                token = firma.dumps(
                    dict(
                        eventi=eventi,
                        id_utente=g.utente['id_utente'],
                        nome_file=secure_filename(file.filename)[:255] or 'eventi.log',
                        contenuto=base64.b64encode(contenuto).decode('ascii'),
                        righe_errate=len(errori),
                        identificativo=secrets.token_hex(32),
                    )
                )
                categorie = visualizzaCategorie()
    except ValueError as e:
        errore = str(e)
    except mysql.connector.Error:
        errore = "Impossibile completare l'operazione sul database. Riprova tra poco."
importati = request.args.get("importati", type=int)
if request.method == "GET" and importati and importati > 0:
    messaggio = f"Importazione completata: {importati} eventi salvati con stato Nuovo."
return render_template(
    "importa_log.html",
    eventi=eventi,
    errori=errori,
    categorie=categorie,
    errore=errore,
    messaggio=messaggio,
    token=token,
)


@app.route("/")
def home():
    return render_template("home.html")


@app.route("/dashboard")
```

```
def dashboard():
    gravita = request.args.get("gravita")
    stato = request.args.get("stato")
    ordine = request.args.get("ordine", "recenti")
    testo = request.args.get("testo", "").strip()

    eventi = visualizzaEventi(gravita, stato, ordine, testo, **ambitoEventi())
    statistiche = statisticheEventi(**ambitoEventi())

    return render_template(
        "eventi.html",
        eventi=eventi,
        gravita=gravita,
        stato=stato,
        ordine=ordine,
        testo_ricerca=testo,
        statistiche=statistiche,
    )


@app.route("/eventi/<int:id_evento>")
def dettagliEvento(id_evento):

    evento = cercaEvento(id_evento, **ambitoEventi())

    if evento is None:
        abort(404)

    return render_template("dettagli_evento.html", evento=evento)


@app.errorhandler(404)
def paginaNonTrovata(errore):
    return render_template("404.html"), 404


@app.route("/eventi/nuovo", methods=["GET", "POST"])
def nuovoEvento():

    errore = None

    if request.method == "POST":
        try:
            id_categoria = int(request.form.get("id_categoria"))
            indirizzo_ip = request.form.get("indirizzo_ip")
            descrizione = request.form.get("descrizione")
            gravita = request.form.get("gravita")

            evento = EventoSicurezza(None, id_categoria, indirizzo_ip, descrizione, gravita)
            inserisciEvento(evento, id_utente=g.utente["id_utente"])
            flash("Evento aggiunto correttamente.", "success")

            return redirect(url_for("dashboard"))

        except (ValueError, TypeError) as e:
            errore = str(e)
        except mysql.connector.Error:
            errore = "Impossibile salvare l'evento. Riprova tra poco."

    categorie = visualizzaCategorie()
    return render_template(
        "nuovo_evento.html", errore=errore, categorie=categorie, dati=request.form
    )


@app.route("/eventi/cerca", methods=["GET"])
def ricercaEvento():

    id_evento = request.args.get("id_evento", type=int)

    if id_evento is None or id_evento <= 0:
        return redirect(url_for("dashboard"))

    evento = cercaEvento(id_evento, **ambitoEventi())
```

```
    if evento is None:
        abort(404)

    return redirect(url_for("dettagliEvento", id_evento=id_evento))


@app.route("/eventi/<int:id_evento>/modifica", methods=["GET", "POST"])
def modificaEventoPagina(id_evento):

    evento = cercaEvento(id_evento, **ambitoEventi())

    if evento is None:
        abort(404)

    categorie = visualizzaCategorie()
    errore = None

    if request.method == "POST":
        try:
            evento_modificato = EventoSicurezza(
                id_evento,
                int(request.form.get("id_categoria")),
                request.form.get("indirizzo_ip"),
                request.form.get("descrizione"),
                request.form.get("gravita"),
                request.form.get("stato"),
                evento["data_evento"],
            )

            modificaEvento(evento_modificato, **ambitoEventi())

            return redirect(url_for("dettagliEvento", id_evento=id_evento))

        except (ValueError, TypeError) as e:
            errore = str(e)
        except mysql.connector.Error:
            errore = "Impossibile salvare le modifiche. Riprova tra poco."

    dati = request.form if request.method == "POST" else evento
    return render_template(
        "modifica_evento.html", evento=evento, categorie=categorie, errore=errore, dati=dati
    )


@app.route("/eventi/<int:id_evento>/elimina", methods=["POST"])
def eliminaEventoPagina(id_evento):

    evento = cercaEvento(id_evento, **ambitoEventi())

    if evento is None:
        abort(404)

    eliminaEvento(id_evento, **ambitoEventi())

    return redirect(url_for("dashboard"))


@app.route("/info")
def info():
    return render_template("info.html")


@app.route("/eventi/esporta")
def esportaEventi():
    gravita = request.args.get("gravita")
    stato = request.args.get("stato")
    ordine = request.args.get("ordine", "recenti")
    testo = request.args.get("testo", "").strip()

    eventi = visualizzaEventi(gravita, stato, ordine, testo, **ambitoEventi())

    file = StringIO(newline="")
    writer = csv.writer(file, delimiter=";")
```

```
    writer.writerow(
        ["ID", "Categoria", "Indirizzo IP", "Descrizione", "GravitÃ ", "Stato", "Data evento"]
    )
    for evento in eventi:
        writer.writerow(
            [
                evento["id_evento"],
                evento["categoria"],
                evento["indirizzo_ip"],
                evento["descrizione"],
                evento["gravita"],
                evento["stato"],
                evento["data_evento"],
            ]
        )
    contenuto = BytesIO(file.getvalue().encode("utf-8-sig"))
    return send_file(
        contenuto, mimetype="text/csv", as_attachment=True, download_name="eventi_sicurezza.csv"
    )


if __name__ == "__main__":
    app.run(debug=True)
```

README.md

426 righe

```
# CS Monitor

**Gestione degli eventi di sicurezza informatica con Python, Flask e MySQL.**

CS Monitor è un'applicazione web che permette di registrare eventi di sicurezza, consultarli in una dashboard e seguirne lo stato di analisi. Gli eventi possono essere inseriti manualmente oppure importati da un file di log. Ogni utente dispone di un account e di uno spazio personale; gli amministratori gestiscono gli accessi e possono consultare tutti gli eventi e le importazioni.

Il progetto nasce come lavoro finale del corso di sviluppo Python ed è ispirato alle attività di un **Security Operations Center (SOC)**: un gruppo che si occupa di monitorare e gestire gli eventi di sicurezza informatica.

**Autore:** Marco Filannino — Studente di Ingegneria Informatica e dell'Automazione presso il Politecnico di Bari.

## Indice

- [Funzionalità](#funzionalità)
- [Account e amministrazione](#account-e-amministrazione)
- [Come funziona](#come-funziona)
- [Tecnologie](#tecnologie)
- [Installazione e avvio](#installazione-e-avvio)
- [Guida all'utilizzo](#guida-allutilizzo)
- [Importazione dei log](#importazione-dei-log)
- [Struttura del progetto](#struttura-del-progetto)
- [Database](#database)
- [Test e verifiche](#test-e-verifiche)
- [Limiti e sviluppi futuri](#limiti-e-sviluppi-futuri)
- [Contatti](#contatti)

## Funzionalità

| Funzione | Cosa permette di fare |
|---|---|
| Registrazione e accesso | Creare un account pubblico e accedere con nome utente o email. |
| Storico personale dei log | Consultare le importazioni e scaricare i file originali. |
| Profilo | Modificare nome utente, email e password. |
| Amministrazione | Creare account, cambiare ruoli, disattivare, riattivare, sbloccare e reimpostare password. |
| Inserimento manuale | Registrare un evento indicando categoria, IP, descrizione e gravità. |
| Dashboard | Consultare gli eventi e il riepilogo di totali, critici, nuovi e risolti. |
| Ricerca | Cercare per ID oppure per testo nell'IP e nella descrizione. |
| Filtri e ordinamento | Filtrare per gravità e stato, ordinando dal più recente o dal più vecchio. |
| Dettagli e modifica | Consultare un evento e aggiornare i suoi dati o il suo stato. |
| Eliminazione | Rimuovere un evento dal database. |
| Importazione dei log | Analizzare un file e assegnare una categoria a ogni evento selezionato nell'anteprima. |
| Esportazione CSV | Scaricare gli eventi rispettando i filtri e l'ordinamento applicati. |

L'interfaccia si adatta alle dimensioni dello schermo e include una pagina di informazioni sul progetto e sullo sviluppatore.
```

Account e amministrazione

- **Registrati** crea un account con ruolo utente. Le password devono avere da 10 a 128 caratteri e vengono conservate co
me hash, mai in chiaro nel database.
- **Accedi** accetta nome utente o email. Dopo cinque password errate l'account resta bloccato per 15 minuti; un amminist
ratore può sbloccarlo.
- **Dashboard**, dettagli, modifica, eliminazione, statistiche ed esportazioni mostrano i propri eventi. Un amministrator
e può consultare e gestire tutti gli eventi.
- **Storico log** conserva file originale, autore, data, dimensione, righe valide, righe errate ed eventi importati. Il d
ownload è riservato al proprietario e agli amministratori.
- **Il mio account** permette di cambiare i dati personali e la password. Il cambio password chiude le sessioni precedent
i su tutti i dispositivi.
- **Gestione account**, disponibile agli amministratori, permette di creare account, assegnare ruoli e gestire l'accesso.
 Una password impostata dall'amministratore è temporanea: l'utente deve cambiarla prima di usare gli eventi. La disattiva
zione conserva lo storico; l'ultimo amministratore attivo non può essere rimosso.

I moduli includono una protezione CSRF. Le anteprime dei log sono firmate e legate all'account che le ha caricate: un alt
ro utente non può confermarle. La stessa anteprima può essere salvata una sola volta. File e relativi eventi vengono regi
strati insieme in una transazione.

Le sessioni scadono dopo due ore di inattività. I cookie sono HttpOnly e SameSite=Lax; quando il sito usa HTTPS imposta `
SESSION_COOKIE_SECURE=true` nel `.env`. Le email servono come identificativo e contatto: non vengono ancora inviate email
 di verifica o recupero password. Per recuperare l'accesso si contatta un amministratore.

Come funziona

Il percorso di un evento inserito manualmente è questo:

```text
L'utente compila il modulo nel browser
                  ↓
Flask riceve i dati
                  ↓
EventoSicurezza controlla che siano validi
                  ↓
Il repository esegue l'inserimento SQL
                  ↓
MySQL conserva l'evento
                  ↓
La dashboard mostra l'evento e la conferma di salvataggio
```

Il codice separa le responsabilità in quattro parti:

- **Interfaccia:** le pagine HTML mostrano i dati e raccolgono le scelte dell'utente.
- **Applicazione:** Flask decide cosa fare quando viene aperta una pagina o inviato un modulo.
- **Modelli e servizi:** le classi controllano i dati; il servizio di importazione interpreta i log.
- **Accesso ai dati:** i repository eseguono le query e gestiscono le connessioni a MySQL.

Questa divisione rende più semplice capire dove intervenire: per cambiare l'aspetto si modificano HTML e CSS; per cambiar
e una query si interviene nel repository.

Tecnologie

Tecnologia	Ruolo nel progetto
Python	Logica applicativa, classi, validazione e analisi dei log.
Flask	Gestione delle pagine e delle richieste web.
MySQL e SQL	Memorizzazione dei dati e operazioni sul database.
HTML e CSS	Struttura e stile delle pagine.
JavaScript	Nome del file scelto e gestione dei menu categoria nell'anteprima.
Bootstrap 5 e Bootstrap Icons	Layout adattabile, componenti grafici e icone.
Jinja2	Generazione delle pagine HTML con i dati ricevuti da Flask.
Git	Gestione delle versioni del progetto.

Le dipendenze Python sono elencate in [`requirements.txt`](requirements.txt). `mysql-connector-python` collega Python a M
ySQL, `python-dotenv` legge il file `.env` e `ItsDangerous` firma i dati dell'anteprima dei log.

Sono utilizzati anche moduli inclusi in Python: `ipaddress` per verificare gli IP, `datetime` per le date, `re` per inter
pretare i log, `csv` per l'esportazione e `unittest` per i test automatici.

Installazione e avvio

1. Prerequisiti

- Python 3.10 o successivo; i test dell'importazione sono stati eseguiti con Python 3.12.
- Un server MySQL avviato e un utente con accesso al database del progetto.
- Connessione Internet per installare le dipendenze e caricare Bootstrap e le icone dai CDN.

I comandi seguenti sono pensati per **Windows PowerShell**, eseguiti dalla cartella del progetto.

2. Prepara l'ambiente Python

```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Se `.venv` esiste già, basta attivarlo e installare le dipendenze mancanti. Quando è attivo, nel terminale compare `(.venv)`.

Se PowerShell impedisce l'attivazione, puoi utilizzare direttamente l'interprete dell'ambiente:

```powershell
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

3. Prepara MySQL

L'applicazione richiede le tabelle `categorie` ed `eventi` e almeno una categoria disponibile. Non le crea automaticamente all'avvio.

Se hai già il database configurato, usa quello esistente. Per un'installazione nuova, trovi uno schema SQL compatibile nella [sezione Database](#database).

4. Configura il file `.env`

Crea un file `.env` nella cartella principale e inserisci i tuoi valori:

```dotenv
DB_HOST=localhost
DB_PORT=3306
DB_USER=tuo_utente_mysql
DB_PASSWORD=tua_password_mysql
DB_NAME=cybersec_monitor
SECRET_KEY=incolla_qui_una_chiave_casuale
```

Per generare la chiave da copiare in `SECRET_KEY`:

```powershell
python -c "import secrets; print(secrets.token_hex(32))"
```

`SECRET_KEY` viene utilizzata per i messaggi di conferma e la firma delle anteprime. Senza una chiave configurata, l'app ne genera una nuova a ogni avvio: le anteprime precedenti diventano quindi non valide.

Il file `.env` è escluso da Git tramite `.gitignore`, perché contiene credenziali personali.

5. Prepara gli account e il primo amministratore

Dopo aver creato le tabelle iniziali `categorie` ed `eventi` e configurato `.env`, esegui:

```powershell
python scripts/setup_account.py --username admin --email tua_email@example.com
```

Lo script aggiunge le tabelle degli account e delle importazioni, conserva gli eventi esistenti e li assegna al primo amministratore. La password iniziale casuale si trova in `instance/credenziali-admin.txt`, escluso da Git. Cambiala dalla pagina del tuo account. Lo script aggiunge una `SECRET_KEY` stabile se manca. Se esiste già un amministratore attivo, la migrazione aggiorna lo schema senza cambiare credenziali.

La migrazione richiede permessi MySQL per creare tabelle, indici e collegamenti. Non viene eseguita automaticamente a ogni richiesta.

6. Avvia l'applicazione

```powershell
python app.py
```

Oppure, senza attivare l'ambiente:

```powershell
.\.venv\Scripts\python.exe app.py
```

Apri http://127.0.0.1:5000 nel browser. Per fermare l'applicazione, premi `Ctrl+C` nel terminale.

L'avvio tramite `app.py` utilizza la modalità debug di Flask, adatta allo sviluppo locale.

Guida all'utilizzo

Registra e gestisci un evento

1. Registrati e accedi, poi apri **Nuovo evento** dal menu.
2. Seleziona una categoria e inserisci IP, descrizione e gravità.
3. Invia il modulo: l'evento viene creato con stato **Nuovo** e con la data e l'ora correnti dell'applicazione.
4. Nella dashboard compare **Evento aggiunto correttamente.**
5. Apri **Dettagli** per consultare l'evento, modificarlo o eliminarlo.

La gravità può essere **Bassa**, **Media**, **Alta** o **Critica**. Lo stato può essere **Nuovo**, **In analisi** o **Risolto**.

Gli stati descrivono l'avanzamento del lavoro; il programma non impone un ordine obbligatorio tra loro.

Cerca ed esporta gli eventi

Nella dashboard puoi combinare ricerca, gravità, stato e ordinamento. Una ricerca composta soltanto dalle cifre 0–9 viene interpretata come ID esatto; gli altri testi vengono cercati nell'IP e nella descrizione.

Le statistiche in alto riguardano **tutti gli eventi del tuo account**, anche quando la tabella è filtrata. Gli amministratori vedono tutti gli eventi degli utenti.

Esporta CSV scarica gli eventi della selezione corrente. Il file usa il punto e virgola come separatore e la codifica UTF-8 con BOM, utile per visualizzare le lettere accentate nei programmi di foglio di calcolo.

Ogni esportazione viene generata in memoria per la singola richiesta, senza sovrascrivere un file condiviso. In caso di errore nei moduli di inserimento e modifica, i valori digitati vengono conservati. Se il database non è disponibile, l'app mostra un messaggio comprensibile.

Importazione dei log

Formato accettato

Ogni riga contiene quattro campi separati da `|`:

```text
data e ora | indirizzo IP | livello | descrizione
```

Esempio:

```text
2026-10-12 09:15:32 | 192.168.1.10 | WARNING | Login fallito per utente admin
2026-10-12 09:17:05 | 192.168.1.20 | ERROR | Connessione bloccata dal firewall
```

La data deve seguire il formato `AAAA-MM-GG HH:MM:SS`. Sono accettati IP sia IPv4 sia IPv6. I livelli devono essere scritti in maiuscolo:

Livello nel log	Gravità assegnata
`INFO`	Bassa
`WARNING`	Media
`ERROR`	Alta
`CRITICAL`	Critica

Procedura

1. Apri **Importa log** e premi **Scegli file**.
2. Seleziona un file `.log` o `.txt` in UTF-8 e premi **Analizza file**.
3. Controlla l'anteprima e le segnalazioni delle righe non valide.
4. Lascia selezionati gli eventi desiderati. Puoi usare **Seleziona tutti** e **Deseleziona tutti**.
5. Scegli una categoria dal menu **Categoria da applicare** e premi **Applica ai selezionati**. Puoi correggere la categoria di ogni singola riga; le righe non selezionate restano invariate.
6. Premi **Importa eventi selezionati**: si apre il dettaglio dell'importazione, con il file originale scaricabile e gli eventi salvati.
7. Puoi ritrovare il file in **Storico log** e gli eventi nella dashboard.

Puoi provare il flusso con [`esempi/eventi.log`](esempi/eventi.log).

Altri file, da 100 a 1000 eventi, sono descritti nella [guida degli esempi](esempi/README.md). Per un file come `firewall.log` puoi assegnare la categoria a tutti i 200 eventi in un solo passaggio.

La preparazione degli account aggiunge anche 30 categorie predefinite per accessi, attacchi di rete, malware, applicazioni e problemi di sistema. Le categorie già presenti e i loro ID vengono conservati; ripetere la preparazione non aggiunge nuovamente gli stessi nomi.

Controlli applicati

- File di massimo **256 KB** e **1000 righe**; le righe vuote vengono ignorate.
- Verifica di formato, data, IP, livello e descrizione.
- Errori accompagnati dal numero di riga, senza impedire l'analisi delle altre righe.
- Categoria obbligatoria soltanto per gli eventi selezionati.
- Anteprima firmata, valida per **30 minuti**; l'analisi non inserisce dati nel database.
- Salvataggio degli eventi selezionati in una singola transazione, con data originale e stato **Nuovo**.

La firma permette di verificare che l'anteprima non sia stata alterata; non cifra i dati. Il salvataggio viene annullato in caso di errore del database se la tabella usa un motore transazionale, come InnoDB.

Struttura del progetto

```text
CS Monitor/
├── app.py                        # Pagine e coordinamento dell'app
├── requirements.txt              # Dipendenze Python
├── database/
│   ├── connessione.py             # Connessione a MySQL
│   ├── categorie_repository.py    # Lettura delle categorie
│   └── eventi_repository.py       # Operazioni e statistiche sugli eventi
├── models/
│   ├── categoria.py               # Classe Categoria e validazione
│   └── evento_sicurezza.py        # Classe EventoSicurezza e validazione
├── services/
│   └── log_importer.py            # Analisi dei log
├── templates/                    # Pagine HTML e componenti comuni
├── static/
│   ├── css/stile.css              # Stile dell'interfaccia
│   └── img/                      # Logo e foto profilo
├── esempi/eventi.log              # File per provare l'importazione
├── instance/                     # Eventuali file locali dell'applicazione
├── test_modelli.py               # Esempi di controllo dei modelli
├── test_database.py              # Verifica della connessione e della lettura
├── test_repository.py            # Verifica CRUD sul database reale
├── test_importazione_log.py       # Test dell'importazione con database simulato
├── routes/account.py             # Accesso, registrazione, profilo e amministrazione
├── services/account.py           # Sessioni, CSRF e controllo del ruolo
├── database/utenti_repository.py # Account e password
├── database/importazioni_repository.py # Storico e file originali
├── database/migrazioni.py        # Aggiornamento dello schema
├── scripts/                      # Preparazione degli account
├── test_account.py               # Test senza database reale
├── test_account_database.py      # Test reali in un database temporaneo
└── test_correzioni.py             # Test su moduli, ricerca, errori e CSV
```

Le operazioni **CRUD** sono le quattro azioni fondamentali sui dati: **Create** (creare), **Read** (leggere), **Update** (modificare) e **Delete** (eliminare).

Database

Il database contiene quattro tabelle. Una categoria può avere più eventi; un utente può avere più eventi e importazioni;

un file importato può avere più eventi. Le categorie sono condivise tra gli account.

Tabella	Campi
`categorie`	`id_categoria`, `nome`
`eventi`	Dati dell’evento, `id_utente` e `id_importazione` (vuoto per gli eventi manuali).
`utenti`	Nome utente, email, hash password, ruolo, stato e informazioni per le sessioni e il blocco degli accessi.
`importazioni`	Proprietario, file originale, data e conteggi dell’importazione.

`id_evento` identifica ogni evento. `id_categoria` nella tabella `eventi` è una **chiave esterna**: collega l'evento a un
a categoria esistente.

Schema per una nuova installazione

Nel repository non è presente un file SQL di inizializzazione. Il seguente schema è compatibile con i campi usati dal cod
ice; non rappresenta un'esportazione del database locale. Eseguilo in un client MySQL su un'installazione nuova:

```sql
CREATE DATABASE cybersec_monitor CHARACTER SET utf8mb4;
USE cybersec_monitor;

CREATE TABLE categorie (
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE eventi (
    id_evento INT AUTO_INCREMENT PRIMARY KEY,
    id_categoria INT NOT NULL,
    indirizzo_ip VARCHAR(45) NOT NULL,
    descrizione TEXT NOT NULL,
    gravita ENUM('Bassa', 'Media', 'Alta', 'Critica') NOT NULL,
    stato ENUM('Nuovo', 'In analisi', 'Risolto') NOT NULL DEFAULT 'Nuovo',
    data_evento DATETIME NOT NULL,
    FOREIGN KEY (id_categoria) REFERENCES categorie(id_categoria)
) ENGINE=InnoDB;

INSERT INTO categorie (nome) VALUES
    ('Accessi sospetti'),
    ('Firewall'),
    ('Malware');
```

Dopo questo schema iniziale esegui `scripts/setup_account.py` per creare `utenti`, `importazioni` e aggiungere i collegam
enti agli eventi. I file originali sono salvati nel database, non in una cartella pubblica.

Le query utilizzano parametri separati dal testo SQL. Le classi verificano gli ID positivi, gli IP, le descrizioni non vu
ote e i valori ammessi per gravità e stato.

Test e verifiche

Test automatici dell'importazione

Con l'ambiente attivo:

```powershell
python -m unittest test_importazione_log -v
```

La suite comprende **13 test** su analisi delle righe, livelli, date, IPv6, limiti dei file, categorie per evento, selezi
oni, anteprime non valide o scadute e gestione degli errori di salvataggio.

Questi test usano un database simulato e **non scrivono nel MySQL reale**. Il loro superamento non sostituisce una verifi
ca dell'applicazione collegata al database.

Per eseguire anche gli 8 test sulle correzioni funzionali (21 test complessivi):

```powershell
python -m unittest test_importazione_log test_correzioni -v
```

Test degli account

```powershell
```

```
python -m unittest test_account test_correzioni test_importazione_log -v
```

Sono 33 test senza scritture sul database reale. Per aggiungere 12 prove complete su MySQL:

```powershell
$env:CS_MONITOR_TEST_DB='1'
python -m unittest test_account test_account_database test_correzioni test_importazione_log -v
Remove-Item Env:CS_MONITOR_TEST_DB
```

Le prove MySQL creano un database temporaneo con nome casuale, lo usano per tutti i dati di prova e lo rimuovono al termine. Richiedono permessi per creare e rimuovere database. Verificano anche l'isolamento tra utenti, lo storico e i download, il doppio invio, il cambio e reset password, la disattivazione e l'annullamento di importazioni fallite.

Riferimenti per le protezioni degli account

- [Flask: sicurezza delle applicazioni web](https://flask.palletsprojects.com/en/stable/web-security/)
- [Flask: autenticazione nel tutorial ufficiale](https://flask.palletsprojects.com/en/stable/tutorial/views/)
- [Werkzeug: hash e verifica delle password](https://werkzeug.palletsprojects.com/en/stable/utils/#werkzeug.security.generate_password_hash)

Controlli sul database reale

```powershell
python test_modelli.py
python test_database.py
python test_repository.py
```

`test_modelli.py` mostra esempi delle classi e delle validazioni. `test_database.py` verifica la connessione e legge gli eventi. `test_repository.py` inserisce, cerca, modifica ed elimina un evento temporaneo: richiede una categoria con **ID 1** e va eseguito su un database di prova.

Problemi comuni

Problema	Cosa controllare
`ModuleNotFoundError`	Usa l'interprete di `.venv` e installa `requirements.txt`.
Errore di connessione a MySQL	Controlla il servizio MySQL e host, porta e credenziali nel `.env`.
Database o tabella non trovati	Prepara lo schema e verifica il valore di `DB_NAME`.
Menu categorie vuoto	Inserisci almeno una categoria nella tabella `categorie`.
Anteprima scaduta o non valida	Ricarica il file; usa una `SECRET_KEY` stabile per mantenere le anteprime tra riavvii.
Stile o icone mancanti	Verifica la connessione ai CDN di Bootstrap e Bootstrap Icons.

Limiti e sviluppi futuri

La versione attuale è pensata per uso didattico e dimostrazioni locali. Gli eventi vengono inseriti dall'utente o ricavati dai file caricati: non è presente un monitoraggio automatico della rete.

Non sono ancora implementati verifica email, recupero password automatico tramite email, gestione delle categorie dall’interfaccia, paginazione degli eventi e riconoscimento dei duplicati tra caricamenti distinti. Confermare due volte la stessa anteprima viene bloccato; caricare nuovamente lo stesso file crea una nuova importazione e nuovi eventi. Il parser accetta soltanto il formato documentato.

Possibili evoluzioni sono grafici nella dashboard, filtri per data e categoria, cronologia delle modifiche e supporto per altri formati di log.

Contatti

Marco Filannino

- [GitHub](https://github.com/MarcoFilannino2004)
- [LinkedIn](https://www.linkedin.com/in/marco-filannino/)
- [Email](mailto:marcofilannino04@gmail.com)

requirements.txt

9 righe

```
blinker==1.9.0
click==8.5.0
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.4
mysql-connector-python==26.7.0
python-dotenv==1.2.4
Werkzeug==3.1.9
```

test_account.py

135 righe

```
"""Test di accesso e protezione dei moduli, senza scrivere sul database."""

import unittest
from unittest.mock import patch
from app import app
from test_support import UTENTE


class AccountTests(unittest.TestCase):
    def setUp(self):
        self.config = app.config.copy()
        app.config.update(TESTING=True, SECRET_KEY='account-test')
        self.addCleanup(lambda: app.config.update(self.config))
        self.client = app.test_client()
        self.lookup = patch('database.utenti_repository.cercaUtente', return_value=dict(UTENTE))
        self.lookup.start()
        self.addCleanup(self.lookup.stop)

    def accedi(self, ruolo='utente', versione=0):
        self.lookup.stop()
        self.lookup = patch(
            'database.utenti_repository.cercaUtente', return_value=dict(UTENTE, ruolo=ruolo)
        )
        self.lookup.start()
        self.addCleanup(self.lookup.stop)
        with self.client.session_transaction() as s:
            s.update(id_utente=1, versione_sessione=versione, csrf_token='token')

    def test_pagine_pubbliche_e_redirect_private(self):
        for url in ('/', '/info', '/login', '/registrazione'):
            self.assertEqual(self.client.get(url).status_code, 200)
        for url in (
            '/dashboard',
            '/eventi/1',
            '/eventi/esporta',
            '/importazioni',
            '/amministrazione/utenti',
        ):
            r = self.client.get(url)
            self.assertEqual(r.status_code, 302)
            self.assertTrue(r.location.endswith('/login'))

    def test_csrf_obbligatorio_anche_registrazione(self):
        with patch('routes.account.creaUtente') as crea:
            self.assertEqual(
                self.client.post('/registrazione', data={'username': 'nuovo'}).status_code, 400
            )
            crea.assert_not_called()

    def test_csrf_non_ascii_restituisce_errore_comprensibile(self):
        self.client.get('/registrazione')
        self.assertEqual(
            self.client.post('/registrazione', data={'csrf_token': 'non-valido-é'}).status_code, 400
        )

    def test_utente_non_puo_amministrare(self):
        self.accedi()
        self.assertEqual(self.client.get('/amministrazione/utenti').status_code, 403)

    def test_registrazione_ignora_ruolo_inviato(self):
```

```
        self.client.get('/registrazione')
        with self.client.session_transaction() as s:
            token = s['csrf_token']
        with patch('routes.account.creaUtente') as crea:
            r = self.client.post(
                '/registrazione',
                data=dict(
                    csrf_token=token,
                    username='nuovo',
                    email='nuovo@example.com',
                    password='una-password-lunga',
                    conferma_password='una-password-lunga',
                    ruolo='admin',
                ),
            )
            self.assertEqual(r.status_code, 303)
            crea.assert_called_once_with('nuovo', 'nuovo@example.com', 'una-password-lunga')

    def test_registrazione_senza_password_non_va_in_errore_interno(self):
        self.client.get('/registrazione')
        with self.client.session_transaction() as s:
            token = s['csrf_token']
        r = self.client.post(
            '/registrazione',
            data=dict(csrf_token=token, username='nuovo', email='nuovo@example.com'),
        )
        self.assertEqual(r.status_code, 200)
        self.assertIn(b'password deve contenere', r.data)

    def test_versione_sessione_revocata(self):
        self.accedi(versione=99)
        self.assertTrue(self.client.get('/dashboard').location.endswith('/login'))

    def test_account_disattivato_perde_accesso(self):
        self.accedi()
        with patch(
            'database.utenti_repository.cercaUtente', return_value=dict(UTENTE, attivo=False)
        ):
            self.assertEqual(self.client.get('/importazioni').status_code, 302)

    def test_logout_solo_post_protetto(self):
        self.accedi()
        self.assertEqual(self.client.get('/logout').status_code, 405)
        self.assertEqual(self.client.post('/logout').status_code, 400)
        self.assertEqual(self.client.post('/logout', data={'csrf_token': 'token'}).status_code, 303)
        with self.client.session_transaction() as s:
            self.assertNotIn('id_utente', s)

    def test_dashboard_e_csv_passano_proprietario(self):
        self.accedi()
        with patch('app.visualizzaEventi', return_value=[]) as elenco, patch(
            'app.statisticheEventi', return_value={}
        ):
            self.client.get('/dashboard')
            self.client.get('/eventi/esporta')
            for chiamata in elenco.call_args_list:
                self.assertEqual(chiamata.kwargs, dict(id_utente=1, amministratore=False))

    def test_admin_puo_consultare_tutto(self):
        self.accedi('admin')
        with patch('app.visualizzaEventi', return_value=[]) as elenco, patch(
            'app.statisticheEventi', return_value={}
        ):
            self.client.get('/dashboard')
            self.assertEqual(elenco.call_args.kwargs, dict(id_utente=1, amministratore=True))

    def test_risposte_private_non_memorizzate(self):
        self.accedi()
        r = self.client.get('/account')
        self.assertEqual(r.headers['Cache-Control'], 'no-store')
        self.assertEqual(r.headers['X-Content-Type-Options'], 'nosniff')


if __name__ == '__main__':
```

```
    unittest.main()
```

test_account_database.py

305 righe

```
"""Prove reali in un database temporaneo; richiedono CS_MONITOR_TEST_DB=1."""

import os
import io
import re
import uuid
import unittest
import mysql.connector
from werkzeug.security import check_password_hash
from app import app
from database.connessione import creaConnessione
from database.migrazioni import preparaAccount
from database.utenti_repository import (
    creaUtente,
    cercaUtente,
    autenticaUtente,
    gestisciUtente,
    reimpostaPassword,
)
from database.eventi_repository import visualizzaEventi
from test_importazione_log import RIGA


@unittest.skipUnless(
    os.getenv('CS_MONITOR_TEST_DB') == '1',
    'Richiede CS_MONITOR_TEST_DB=1 e MySQL con permesso di creare database di prova.',
)
class AccountDatabaseTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.originale = os.getenv('DB_NAME')
        cls.nome = 'cs_monitor_test_' + uuid.uuid4().hex
        cls.config = app.config.copy()
        app.config.update(TESTING=True, SECRET_KEY='test-database')
        cls.server = mysql.connector.connect(
            host=os.getenv('DB_HOST'),
            port=int(os.getenv('DB_PORT', '3306')),
            user=os.getenv('DB_USER'),
            password=os.getenv('DB_PASSWORD'),
        )
        c = cls.server.cursor()
        c.execute(f'CREATE DATABASE `{cls.nome}` CHARACTER SET utf8mb4')
        c.execute(f'USE `{cls.nome}`')
        c.execute(
            'CREATE TABLE categorie (id_categoria INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(100)) ENGINE=InnoDB'
        )
        c.execute(
            "CREATE TABLE eventi (id_evento INT AUTO_INCREMENT PRIMARY KEY, id_categoria INT NOT NULL, indirizzo_ip VARCH
AR(45), descrizione TEXT, gravita VARCHAR(20), stato VARCHAR(20), data_evento DATETIME, FOREIGN KEY (id_categoria) REFERE
NCES categorie(id_categoria)) ENGINE=InnoDB"
        )
        c.execute("INSERT INTO categorie (nome) VALUES ('Accessi')")
        cls.server.commit()
        c.close()
        os.environ['DB_NAME'] = cls.nome
        preparaAccount()
        preparaAccount()  # La migrazione deve poter essere ripetuta.
        cls.admin = creaUtente('admin_test', 'admin@test.example', 'password-admin-test', 'admin')

    @classmethod
    def tearDownClass(cls):
        os.environ['DB_NAME'] = cls.originale
        app.config.update(cls.config)
        # Il nome generato è verificato prima della rimozione del solo database di prova.
        assert re.fullmatch(r'cs_monitor_test_[a-f0-9]{32}', cls.nome)
        c = cls.server.cursor()
        c.execute(f'DROP DATABASE `{cls.nome}`')
        c.close()
        cls.server.close()
```

```
    def setUp(self):
        identita = uuid.uuid4().hex[:12]
        self.username = 'u_' + identita
        self.password = 'password-di-prova-123'
        self.id = creaUtente(self.username, identita + '@test.example', self.password)
        self.client = app.test_client()
        self.login(self.client, self.username, self.password)

    def token(self, client=None, url='/account'):
        client = client or self.client
        client.get(url)
        with client.session_transaction() as s:
            return s['csrf_token']

    def login(self, client, username, password):
        return client.post(
            '/login',
            data=dict(
                csrf_token=self.token(client, '/login'), identificativo=username, password=password
            ),
        )

    def post(self, url, data, client=None):
        client = client or self.client
        data['csrf_token'] = self.token(client)
        return client.post(url, data=data)

    def evento(self):
        r = self.post(
            '/eventi/nuovo',
            dict(
                id_categoria='1',
                indirizzo_ip='192.0.2.1',
                descrizione='Evento privato',
                gravita='Alta',
            ),
        )
        self.assertEqual(r.status_code, 302)
        return visualizzaEventi(id_utente=self.id)[0]['id_evento']

    def importazione(self):
        r = self.post('/eventi/importa', {'file_log': (io.BytesIO(RIGA.encode()), 'prova.log')})
        token = re.search(r'name="anteprima" value="([^"]+)"', r.get_data(as_text=True)).group(1)
        dati = dict(azione='conferma', anteprima=token, selezionati='1', categoria_1='1')
        r = self.post('/eventi/importa', dict(dati))
        self.assertEqual(r.status_code, 303)
        return r.location, dati

    def test_password_hash_e_login_email(self):
        u = cercaUtente(self.id)
        self.assertNotEqual(u['password_hash'], self.password)
        self.assertTrue(check_password_hash(u['password_hash'], self.password))
        self.assertEqual(autenticaUtente(u['email'], self.password)['id_utente'], self.id)

    def test_repository_senza_proprietario_non_espone_eventi(self):
        from database.eventi_repository import cercaEvento, eliminaEvento, statisticheEventi

        evento = self.evento()
        self.assertEqual(visualizzaEventi(), [])
        self.assertIsNone(cercaEvento(evento))
        self.assertFalse(eliminaEvento(evento))
        self.assertEqual(statisticheEventi()['totale'], 0)
        self.assertEqual(self.client.get(f'/eventi/{evento}').status_code, 200)

    def test_categorie_predefinite_conservano_esistenti_e_non_duplicano(self):
        from database.categorie_repository import (
            aggiungiCategoriePredefinite,
            visualizzaCategorie,
            CATEGORIE_PREDEFINITE,
        )

        prima = {c['id_categoria']: c['nome'] for c in visualizzaCategorie()}
        aggiungiCategoriePredefinite()
        dopo = {c['id_categoria']: c['nome'] for c in visualizzaCategorie()}
```

```
        for id_categoria, nome in prima.items():
            self.assertEqual(dopo[id_categoria], nome)
        self.assertTrue(set(CATEGORIE_PREDEFINITE).issubset(set(dopo.values())))
        self.assertEqual(aggiungiCategoriePredefinite(), 0)
        self.assertEqual(len(visualizzaCategorie()), len(dopo))

    def test_isolamento_lettura_modifica_elimina_csv_statistiche(self):
        evento = self.evento()
        altro = creaUtente(
            'altro_' + uuid.uuid4().hex[:8], uuid.uuid4().hex + '@test.example', self.password
        )
        client = app.test_client()
        u = cercaUtente(altro)
        self.login(client, u['username'], self.password)
        for suffix in ('', '/modifica'):
            self.assertEqual(client.get(f'/eventi/{evento}{suffix}').status_code, 404)
        self.assertEqual(self.post(f'/eventi/{evento}/elimina', {}, client).status_code, 404)
        self.assertEqual(
            self.post(
                f'/eventi/{evento}/modifica',
                dict(
                    id_categoria=1,
                    indirizzo_ip='192.0.2.2',
                    descrizione='Tentativo',
                    gravita='Bassa',
                    stato='Nuovo',
                ),
                client,
            ).status_code,
            404,
        )
        self.assertNotIn(b'Evento privato', client.get('/eventi/esporta').data)
        self.assertNotIn(b'Evento privato', client.get('/dashboard').data)
        self.assertEqual(len(visualizzaEventi(id_utente=altro)), 0)
        self.assertEqual(self.client.get(f'/eventi/{evento}').status_code, 200)

    def test_importazione_storico_download_e_doppio_invio(self):
        url, dati = self.importazione()
        self.assertIn(b'Importazione completata', self.client.get(url).data)
        self.assertEqual(self.client.get(url + '/file').data, RIGA.encode())
        self.assertIn(b'prova.log', self.client.get('/importazioni').data)
        self.assertIn(b'Login fallito per utente admin', self.client.get(url).data)
        r = self.post('/eventi/importa', dati)
        self.assertIn('già stato importato', r.get_data(as_text=True))
        self.assertEqual(len(visualizzaEventi(id_utente=self.id)), 1)
        estraneo = app.test_client()
        self.login(estraneo, 'admin_test', 'password-admin-test')
        self.assertEqual(estraneo.get(url + '/file').status_code, 200)
        altro = creaUtente(
            'estraneo_' + uuid.uuid4().hex[:8], uuid.uuid4().hex + '@test.example', self.password
        )
        estraneo = app.test_client()
        self.login(estraneo, cercaUtente(altro)['username'], self.password)
        self.assertEqual(estraneo.get(url).status_code, 404)
        self.assertEqual(estraneo.get(url + '/file').status_code, 404)
        r = self.post('/eventi/importa', dict(dati), estraneo)
        self.assertIn(b'altro account', r.data)
        self.assertEqual(len(visualizzaEventi(id_utente=altro)), 0)

    def test_cambio_password_revoca_sessioni(self):
        secondo = app.test_client()
        self.login(secondo, self.username, self.password)
        r = self.post(
            '/account',
            dict(
                azione='password',
                password_attuale=self.password,
                nuova_password='nuova-password-1234',
                conferma_password='nuova-password-1234',
            ),
        )
        self.assertEqual(r.status_code, 303)
        self.assertEqual(secondo.get('/dashboard').status_code, 302)
        self.assertIsNone(autenticaUtente(self.username, self.password))
```

```
        self.assertIsNotNone(autenticaUtente(self.username, 'nuova-password-1234'))

    def test_disattivazione_riattivazione_conserva_dati(self):
        evento = self.evento()
        gestisciUtente(self.id, self.admin, 'utente', False)
        self.assertEqual(self.client.get('/dashboard').status_code, 302)
        self.assertIsNone(autenticaUtente(self.username, self.password))
        gestisciUtente(self.id, self.admin, 'utente', True)
        self.login(self.client, self.username, self.password)
        self.assertEqual(self.client.get(f'/eventi/{evento}').status_code, 200)

    def test_blocco_e_sblocco_login(self):
        for _ in range(5):
            self.assertIsNone(autenticaUtente(self.username, 'password-errata'))
        self.assertIsNone(autenticaUtente(self.username, self.password))
        gestisciUtente(self.id, self.admin, 'utente', True)
        self.assertIsNotNone(autenticaUtente(self.username, self.password))

    def test_reset_password_richiede_cambio_e_revoca_sessioni(self):
        reimpostaPassword(self.id, self.admin, 'password-temporanea-123')
        self.assertEqual(self.client.get('/dashboard').status_code, 302)
        self.assertEqual(
            self.login(self.client, self.username, 'password-temporanea-123').status_code, 303
        )
        self.assertTrue(self.client.get('/dashboard').location.endswith('/account'))
        self.assertIn(b'password temporanea', self.client.get('/account').data)
        r = self.post(
            '/account',
            dict(
                azione='password',
                password_attuale='password-temporanea-123',
                nuova_password='password-scelta-utente-123',
                conferma_password='password-scelta-utente-123',
            ),
        )
        self.assertEqual(r.status_code, 303)
        self.login(self.client, self.username, 'password-scelta-utente-123')
        self.assertEqual(self.client.get('/dashboard').status_code, 200)

    def test_importazione_fallita_annulla_file_ed_eventi(self):
        from database.importazioni_repository import salvaImportazione, visualizzaImportazioni
        from models.evento_sicurezza import EventoSicurezza

        e = EventoSicurezza(None, 99999, '192.0.2.1', 'Categoria inesistente', 'Alta')
        with self.assertRaises(mysql.connector.IntegrityError):
            salvaImportazione([e], self.id, 'prova.log', RIGA.encode(), 1, 0, uuid.uuid4().hex)
        self.assertEqual(visualizzaImportazioni(self.id), [])
        self.assertEqual(visualizzaEventi(id_utente=self.id), [])

    def test_admin_protetto_e_modifica_ruolo(self):
        with self.assertRaises(ValueError):
            gestisciUtente(self.admin, self.admin, 'utente', False)
        gestisciUtente(self.id, self.admin, 'admin', True)
        self.login(self.client, self.username, self.password)
        self.assertEqual(self.client.get('/amministrazione/utenti').status_code, 200)
        gestisciUtente(self.id, self.admin, 'utente', True)
        self.login(self.client, self.username, self.password)
        self.assertEqual(self.client.get('/amministrazione/utenti').status_code, 403)

    def test_registrazione_pubblica_duplicati_e_profilo(self):
        client = app.test_client()
        nome = 'registrato_' + uuid.uuid4().hex[:8]
        dati = dict(
            username=nome,
            email=nome + '@test.example',
            password=self.password,
            conferma_password=self.password,
            ruolo='admin',
        )
        dati['csrf_token'] = self.token(client, '/registrazione')
        self.assertEqual(client.post('/registrazione', data=dati).status_code, 303)
        self.assertEqual(autenticaUtente(nome, self.password)['ruolo'], 'utente')
        self.assertIn(
            'già utilizzati', client.post('/registrazione', data=dati).get_data(as_text=True)
```

```
        )
        r = self.post(
            '/account',
            dict(
                azione='profilo',
                username=self.username + '_edit',
                email='edit_' + uuid.uuid4().hex + '@test.example',
            ),
        )
        self.assertEqual(r.status_code, 303)
        self.assertEqual(cercaUtente(self.id)['username'], self.username + '_edit')


if __name__ == '__main__':
    unittest.main()
```

test_correzioni.py

143 righe

```
import csv
import io
import unittest
from datetime import datetime
from unittest.mock import patch, MagicMock
import mysql.connector
from app import app
from test_support import preparaClient
from database.eventi_repository import visualizzaEventi


class CorrezioniTests(unittest.TestCase):
    def setUp(self):
        self.config = app.config.copy()
        app.config.update(TESTING=True, SECRET_KEY="test-correzioni")
        self.client = preparaClient(self, app)
        self.categorie = [
            {"id_categoria": 1, "nome": "Accessi"},
            {"id_categoria": 2, "nome": "Firewall"},
        ]
        self.evento = dict(
            id_evento=1,
            id_categoria=1,
            categoria="Accessi",
            indirizzo_ip="192.0.2.1",
            descrizione="Originale",
            gravita="Media",
            stato="Nuovo",
            data_evento=datetime(2026, 10, 12),
        )
        self.dati = dict(
            id_categoria="2",
            indirizzo_ip="errato",
            descrizione="Bozza da conservare",
            gravita="Alta",
            stato="In analisi",
        )
        self.cat_patch = patch("app.visualizzaCategorie", return_value=self.categorie)
        self.cat_patch.start()

    def tearDown(self):
        self.cat_patch.stop()
        app.config.clear()
        app.config.update(self.config)

    def verifica_dati(self, risposta):
        html = risposta.get_data(as_text=True)
        self.assertIn('value="errato"', html)
        self.assertIn('Bozza da conservare', html)
        self.assertIn('value="2" selected', html)
        self.assertIn('value="Alta" selected', html)

    @patch("app.inserisciEvento")
    def test_nuovo_conserva_dati_con_errore(self, salva):
        self.verifica_dati(self.client.post('/eventi/nuovo', data=self.dati))
        salva.assert_not_called()
```

```
    @patch("app.modificaEvento")
    def test_modifica_conserva_dati_con_errore(self, salva):
        with patch("app.cercaEvento", return_value=self.evento):
            risposta = self.client.post('/eventi/1/modifica', data=self.dati)
        self.verifica_dati(risposta)
        self.assertIn('value="In analisi" selected', risposta.get_data(as_text=True))
        salva.assert_not_called()

    def test_modifica_get_mostra_valori_originali(self):
        with patch("app.cercaEvento", return_value=self.evento):
            html = self.client.get('/eventi/1/modifica').get_data(as_text=True)
        self.assertIn('Originale', html)
        self.assertIn('value="1" selected', html)
        self.assertIn('value="Media" selected', html)

    def test_errori_database_gestiti(self):
        for target, url in [
            ('app.visualizzaEventi', '/dashboard'),
            ('app.cercaEvento', '/eventi/1'),
            ('app.visualizzaCategorie', '/eventi/nuovo'),
            ('app.visualizzaEventi', '/eventi/esporta'),
        ]:
            with self.subTest(url=url), patch(
                target, side_effect=mysql.connector.Error('dettaglio riservato')
            ):
                risposta = self.client.get(url)
                self.assertEqual(risposta.status_code, 503)
                self.assertIn(b'Servizio temporaneamente non disponibile', risposta.data)
                self.assertNotIn(b'dettaglio riservato', risposta.data)

    def test_errore_salvataggio_conserva_modulo(self):
        dati = dict(self.dati, indirizzo_ip='192.0.2.2')
        with patch('app.inserisciEvento', side_effect=mysql.connector.Error('dettaglio riservato')):
            risposta = self.client.post('/eventi/nuovo', data=dati)
        self.assertIn(b'Bozza da conservare', risposta.data)
        self.assertIn(b'Riprova tra poco', risposta.data)
        self.assertNotIn(b'dettaglio riservato', risposta.data)

    @patch('database.eventi_repository.creaConnessione')
    def test_ricerche_numeriche_e_unicode(self, crea):
        cursore = MagicMock()
        cursore.fetchall.return_value = []
        crea.return_value.cursor.return_value = cursore
        for testo, atteso in [
            ('²', ('%²%', '%²%')),
            ('0012', (12,)),
            ('9' * 5000, ()),
            ('0' * 5000 + '12', (12,)),
        ]:
            with self.subTest(testo=testo[:20]):
                visualizzaEventi(testo=testo, amministratore=True)
                query, valori = cursore.execute.call_args.args
                self.assertEqual(valori, atteso)
                if testo == '9' * 5000:
                    self.assertIn('AND 1=0', query)

    def test_esportazioni_indipendenti_e_filtrate(self):
        def elenco(gravita, stato, ordine, testo, **ambito):
            return [dict(self.evento, descrizione=testo)]

        with patch('app.visualizzaEventi', side_effect=elenco) as leggi, patch(
            'builtins.open', side_effect=AssertionError('Scrittura su disco non prevista')
        ):
            prima = self.client.get('/eventi/esporta?testo=prima&gravita=Alta', buffered=False)
            seconda = self.client.get('/eventi/esporta?testo=seconda', buffered=False)
            for risposta, atteso in [(prima, 'prima'), (seconda, 'seconda')]:
                righe = list(
                    csv.reader(io.StringIO(risposta.data.decode('utf-8-sig')), delimiter=';')
                )
                self.assertEqual(righe[1][3], atteso)
                self.assertIn('attachment', risposta.headers['Content-Disposition'])
                risposta.close()
            self.assertEqual(leggi.call_args_list[0].args, ('Alta', None, 'recenti', 'prima'))
```

```
    def test_html_unico_e_tabella_vuota(self):
        with patch('app.visualizzaEventi', return_value=[]), patch(
            'app.statisticheEventi', return_value=dict(totale=0)
        ):
            html = self.client.get('/dashboard').get_data(as_text=True)
        self.assertEqual(html.count('<html'), 1)
        self.assertEqual(html.count('<head>'), 1)
        self.assertIn('colspan="7"', html)


if __name__ == '__main__':
    unittest.main()
```

test_database.py

38 righe

```
from database.connessione import creaConnessione
import mysql.connector

connessione = None
cursore = None

try:
    connessione = creaConnessione()
    cursore = connessione.cursor(dictionary=True)

    print("Connessione a MySQL riuscita!")

    query = """
        SELECT e.id_evento, c.nome AS categoria, e.indirizzo_ip,
               e.descrizione, e.gravita, e.stato, e.data_evento
        FROM eventi e
        JOIN categorie c ON e.id_categoria = c.id_categoria
        ORDER BY e.id_evento
    """

    cursore.execute(query)
    eventi = cursore.fetchall()

    for evento in eventi:
        print(
            f"{evento['id_evento']} | {evento['categoria']} | {evento['indirizzo_ip']} | {evento['gravita']} | {evento['s
tato']}"
        )

except mysql.connector.Error as errore:
    print(f"Errore MySQL: {errore}")

finally:
    if cursore is not None:
        cursore.close()

    if connessione is not None and connessione.is_connected():
        connessione.close()
        print("Connessione chiusa.")
```

test_importazione_log.py

223 righe

```
import io
import base64
from test_support import preparaClient
import unittest
from datetime import datetime
from unittest.mock import patch, MagicMock
from itsdangerous import URLSafeTimedSerializer, SignatureExpired
import mysql.connector

from app import app
from services.log_importer import analizzaLog, MAX_BYTES
from database.eventi_repository import inserisciEventi
from models.evento_sicurezza import EventoSicurezza

RIGA = "2026-10-12 09:15:32 | 192.168.1.10 | WARNING | Login fallito per utente admin"
```

```
class ParserTests(unittest.TestCase):
    def test_livelli_date_e_ipv6(self):
        testo = "\n".join(
            RIGA.replace("WARNING", livello).replace("192.168.1.10", "2001:db8::1")
            for livello in ("INFO", "WARNING", "ERROR", "CRITICAL")
        )
        eventi, errori = analizzaLog(testo.encode("utf-8-sig"))
        self.assertEqual(errori, [])
        self.assertEqual([e["gravita"] for e in eventi], ["Bassa", "Media", "Alta", "Critica"])
        self.assertEqual(eventi[0]["data_evento"], "2026-10-12 09:15:32")

    def test_errori_e_numeri_riga(self):
        righe = [
            RIGA,
            "",
            "riga errata",
            RIGA.replace("192.168.1.10", "999.1.1.1"),
            RIGA.replace("2026-10-12", "2026-02-30"),
            RIGA.replace("WARNING", "DEBUG"),
            RIGA.rsplit("|", 1)[0] + "|   ",
        ]
        eventi, errori = analizzaLog("\n".join(righe).encode())
        self.assertEqual(len(eventi), 1)
        self.assertEqual([e["riga"] for e in errori], [3, 4, 5, 6, 7])

    def test_limiti_e_codifica(self):
        for contenuto in (b"", b"\xff", b"a" * (MAX_BYTES + 1), b"\n" * 1001):
            with self.subTest(contenuto=contenuto[:20]), self.assertRaises(ValueError):
                analizzaLog(contenuto)


class RouteTests(unittest.TestCase):
    def setUp(self):
        app.config.update(TESTING=True, SECRET_KEY="chiave-test")
        self.client = preparaClient(self, app)
        self.eventi, _ = analizzaLog((RIGA + "\n" + RIGA.replace("WARNING", "ERROR")).encode())
        self.token = URLSafeTimedSerializer(
            app.config["SECRET_KEY"], salt="importazione-log"
        ).dumps(
            dict(
                eventi=self.eventi,
                id_utente=1,
                nome_file='eventi.log',
                contenuto=base64.b64encode(
                    (RIGA + "\n" + RIGA.replace('WARNING', 'ERROR')).encode()
                ).decode(),
                righe_errate=0,
                identificativo='test-anteprima',
            )
        )

    @patch("app.salvaImportazione", return_value=1)
    @patch("app.visualizzaCategorie", return_value=[{"id_categoria": 2, "nome": "Accessi"}])
    def test_anteprima_non_salva(self, categorie, salva):
        risposta = self.client.post(
            "/eventi/importa", data={"file_log": (io.BytesIO(RIGA.encode()), "eventi.log")}
        )
        self.assertEqual(risposta.status_code, 200)
        self.assertIn(b"anteprima", risposta.data)
        salva.assert_not_called()

    @patch("app.salvaImportazione", return_value=1)
    @patch("app.visualizzaCategorie", return_value=[{"id_categoria": 2, "nome": "Accessi"}])
    def test_importa_solo_selezionati(self, categorie, salva):
        risposta = self.client.post(
            "/eventi/importa",
            data={
                "azione": "conferma",
                "anteprima": self.token,
                "selezionati": "2",
                "categoria_2": "2",
            },
        )
```

```
        self.assertEqual(risposta.status_code, 303)
        eventi = salva.call_args.args[0]
        self.assertEqual(len(eventi), 1)
        self.assertEqual(eventi[0].id_categoria, 2)
        self.assertEqual(eventi[0].gravita, "Alta")
        self.assertEqual(eventi[0].stato, "Nuovo")
        self.assertEqual(eventi[0].data_evento, datetime(2026, 10, 12, 9, 15, 32))
        self.assertIn("/importazioni/1", risposta.location)

    @patch("app.salvaImportazione", return_value=1)
    @patch("app.visualizzaCategorie", return_value=[{"id_categoria": 2, "nome": "Accessi"}])
    def test_conferme_non_valide(self, categorie, salva):
        casi = [
            ("alterato", "1", "2"),
            (self.token, "999", "2"),
            (self.token, "1", "999"),
            (self.token, "", "2"),
        ]
        for token, riga, categoria in casi:
            with self.subTest(riga=riga, categoria=categoria):
                risposta = self.client.post(
                    "/eventi/importa",
                    data={
                        "azione": "conferma",
                        "anteprima": token,
                        "selezionati": riga,
                        "categoria_1": categoria,
                    },
                )
                self.assertEqual(risposta.status_code, 200)
                self.assertIn(b"alert-danger", risposta.data)
        salva.assert_not_called()

    @patch("app.salvaImportazione", return_value=1)
    @patch(
        "app.visualizzaCategorie",
        return_value=[
            {"id_categoria": 2, "nome": "Accessi"},
            {"id_categoria": 3, "nome": "Firewall"},
        ],
    )
    def test_categorie_diverse_per_evento(self, categorie, salva):
        risposta = self.client.post(
            "/eventi/importa",
            data={
                "azione": "conferma",
                "anteprima": self.token,
                "selezionati": ["1", "2"],
                "categoria_1": "2",
                "categoria_2": "3",
            },
        )
        self.assertEqual(risposta.status_code, 303)
        self.assertEqual([e.id_categoria for e in salva.call_args.args[0]], [2, 3])

    @patch("app.salvaImportazione", return_value=1)
    @patch("app.visualizzaCategorie", return_value=[{"id_categoria": 2, "nome": "Accessi"}])
    def test_categoria_mancante_blocca_intera_importazione(self, categorie, salva):
        risposta = self.client.post(
            "/eventi/importa",
            data={
                "azione": "conferma",
                "anteprima": self.token,
                "selezionati": ["1", "2"],
                "categoria_1": "2",
            },
        )
        self.assertIn(b"categoria valida per la riga 2", risposta.data)
        self.assertIn(b'value="2" selected', risposta.data)
        salva.assert_not_called()

    @patch("app.URLSafeTimedSerializer")
    def test_anteprima_scaduta(self, firma):
        firma.return_value.loads.side_effect = SignatureExpired("scaduta")
```

```
        risposta = self.client.post(
            "/eventi/importa", data={"azione": "conferma", "anteprima": self.token}
        )
        self.assertIn(b"scaduta", risposta.data)

    def test_vecchia_anteprima_chiede_nuovo_caricamento(self):
        token = URLSafeTimedSerializer(app.config['SECRET_KEY'], salt='importazione-log').dumps(
            self.eventi
        )
        r = self.client.post('/eventi/importa', data=dict(azione='conferma', anteprima=token))
        self.assertEqual(r.status_code, 200)
        self.assertIn(b'Carica nuovamente', r.data)

    def test_upload_non_valido(self):
        self.assertEqual(self.client.get("/eventi/importa").status_code, 200)
        for data in (
            {},
            {"file_log": (io.BytesIO(b"test"), "file.csv")},
            {"file_log": (io.BytesIO(b"x" * (MAX_BYTES + 1)), "file.log")},
        ):
            risposta = self.client.post("/eventi/importa", data=data)
            self.assertIn(b"alert-danger", risposta.data)

    @patch("app.visualizzaCategorie", return_value=[{"id_categoria": 2, "nome": "Accessi"}])
    @patch("app.salvaImportazione", side_effect=mysql.connector.Error("errore interno"))
    def test_errore_database(self, salva, categorie):
        risposta = self.client.post(
            "/eventi/importa",
            data={
                "azione": "conferma",
                "anteprima": self.token,
                "selezionati": "1",
                "categoria_1": "2",
            },
        )
        self.assertIn(b"Riprova tra poco", risposta.data)
        self.assertNotIn(b"errore interno", risposta.data)


class RepositoryTests(unittest.TestCase):
    @patch("database.eventi_repository.creaConnessione")
    def test_transazione_e_rollback(self, crea):
        connessione = MagicMock()
        crea.return_value = connessione
        evento = EventoSicurezza(None, 2, "192.168.1.10", "Test", "Media")
        inserisciEventi([evento])
        connessione.commit.assert_called_once()
        connessione.rollback.assert_not_called()
        connessione.reset_mock()
        connessione.cursor.return_value.executemany.side_effect = mysql.connector.Error("errore")
        with self.assertRaises(mysql.connector.Error):
            inserisciEventi([evento])
        connessione.commit.assert_not_called()
        connessione.rollback.assert_called_once()
        connessione.close.assert_called_once()


if __name__ == "__main__":
    unittest.main()
```

test_modelli.py

16 righe

```
from models.categoria import Categoria
from models.evento_sicurezza import EventoSicurezza

categoria = Categoria(1, "Login fallito")
print(categoria)

evento = EventoSicurezza(None, 1, "192.168.1.15", "Tentativi di accesso falliti", "Alta")
print(evento)

evento.stato = "In analisi"
print(evento)
```

```
try:
    evento = EventoSicurezza(None, 1, "999.999.999.999", "Tentativo di accesso", "Alta")
except ValueError as errore:
    print(f"Errore rilevato correttamente: {errore}")
```

test_repository.py

88 righe

```
from database.eventi_repository import (
    inserisciEvento,
    visualizzaEventi,
    cercaEvento,
    modificaEvento,
    eliminaEvento,
)
from models.evento_sicurezza import EventoSicurezza


def stampaEventi():
    print("\n========== ELENCO EVENTI ==========")

    for evento in visualizzaEventi(amministratore=True):
        print(
            f"{evento['id_evento']} | {evento['categoria']} | {evento['indirizzo_ip']} | {evento['gravita']} | {evento['s
tato']}"
        )


def testCRUD():
    id_test = None

    try:
        # READ
        stampaEventi()

        # CREATE
        print("\n========== CREATE ==========")

        nuovo_evento = EventoSicurezza(
            None, 1, "192.168.1.200", "Evento temporaneo per test CRUD", "Media"
        )
        id_test = inserisciEvento(nuovo_evento)

        print(f"Evento creato con ID: {id_test}")

        # READ - Singolo evento
        print("\n========== READ ==========")

        evento_trovato = cercaEvento(id_test, amministratore=True)
        assert evento_trovato is not None, "Evento non trovato dopo l'inserimento."

        print(evento_trovato)

        # UPDATE
        print("\n========== UPDATE ==========")

        nuovo_evento.descrizione = "Evento modificato durante il test CRUD"
        nuovo_evento.gravita = "Alta"
        nuovo_evento.stato = "In analisi"

        modificato = modificaEvento(nuovo_evento, amministratore=True)
        assert modificato, "La modifica non Ã¨ stata eseguita."

        evento_aggiornato = cercaEvento(id_test, amministratore=True)
        assert evento_aggiornato["descrizione"] == "Evento modificato durante il test CRUD"
        assert evento_aggiornato["gravita"] == "Alta"
        assert evento_aggiornato["stato"] == "In analisi"

        print("Evento modificato correttamente!")
        print(evento_aggiornato)

        # DELETE
        print("\n========== DELETE ==========")

        eliminato = eliminaEvento(id_test, amministratore=True)
```

```
        assert eliminato, "L'eliminazione non Ã¨ stata eseguita."

        assert (
            cercaEvento(id_test, amministratore=True) is None
        ), "L'evento risulta ancora presente."

        id_test = None
        print("Evento eliminato correttamente!")

        # READ finale
        stampaEventi()

        print("\nTUTTI I TEST CRUD SUPERATI!")

    finally:
        if id_test is not None and cercaEvento(id_test, amministratore=True) is not None:
            eliminaEvento(id_test, amministratore=True)
            print("\nPulizia: evento temporaneo eliminato.")


if __name__ == "__main__":
    testCRUD()
```

test_support.py

28 righe

```
from unittest.mock import patch

UTENTE = dict(
    id_utente=1,
    username='utente_test',
    email='test@example.com',
    ruolo='utente',
    attivo=True,
    versione_sessione=0,
)


def preparaClient(test, app):
    test.utente_patch = patch('database.utenti_repository.cercaUtente', return_value=UTENTE)
    test.utente_patch.start()
    test.addCleanup(test.utente_patch.stop)
    client = app.test_client()
    with client.session_transaction() as session:
        session.update(id_utente=1, versione_sessione=0, csrf_token='csrf-test')
    originale = client.post

    def post(url, **kwargs):
        dati = dict(kwargs.pop('data', {}))
        dati.setdefault('csrf_token', 'csrf-test')
        return originale(url, data=dati, **kwargs)

    client.post = post
    return client
```

database/categorie_repository.py

72 righe

```
import mysql.connector
from database.connessione import creaConnessione
from database.strumenti import apriCursore

CATEGORIE_PREDEFINITE = (
    'Autenticazione e accessi',
    'Tentativi di forza bruta',
    'Account bloccati',
    'Accessi non autorizzati',
    'Permessi e privilegi',
    'Gestione delle sessioni',
    'Firewall e traffico bloccato',
    'Scansione delle porte',
    'Traffico di rete anomalo',
    'Attacchi DoS e DDoS',
    'Connessioni VPN',
    'Errori DNS',
    'Certificati e connessioni TLS',
```

```
    'Malware e antivirus',
    'Ransomware',
    'Phishing e posta sospetta',
    'File sospetti e quarantena',
    'Integrità dei file',
    'Esfiltrazione dei dati',
    'Errori applicativi',
    'Errori del database',
    'Servizi non disponibili',
    'Errori del sistema operativo',
    'Spazio disco e archiviazione',
    'Prestazioni e risorse',
    'Backup e ripristino',
    'Aggiornamenti di sicurezza',
    'Configurazioni non autorizzate',
    'Dispositivi e hardware',
    'Monitoraggio e audit',
)


def aggiungiCategoriePredefinite():
    """Aggiunge soltanto i nomi mancanti, mantenendo categorie e ID esistenti."""
    with apriCursore() as (connessione, cursore):
        cursore.execute('SELECT nome FROM categorie FOR UPDATE')
        esistenti = {riga[0].strip().casefold() for riga in cursore.fetchall()}
        nuove = [(nome,) for nome in CATEGORIE_PREDEFINITE if nome.casefold() not in esistenti]
        if nuove:
            cursore.executemany('INSERT INTO categorie (nome) VALUES (%s)', nuove)
        connessione.commit()
        return len(nuove)


def visualizzaCategorie():
    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor(dictionary=True)

        query = "SELECT id_categoria, nome FROM categorie ORDER BY nome"

        cursore.execute(query)
        return cursore.fetchall()

    except mysql.connector.Error as errore:
        print(f"Errore durante la lettura delle categorie: {errore}")
        raise

    finally:
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()
```

database/connessione.py

18 righe

```
import os
import mysql.connector
from dotenv import load_dotenv

# Carico le variabili d'ambiente dal file .env
load_dotenv()


def creaConnessione():
    connessione = mysql.connector.connect(
        host=os.getenv("DB_HOST"),
        port=int(os.getenv("DB_PORT", "3306")),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME"),
    )
```

```
    return connessione
```

database/eventi_repository.py

298 righe

```
import mysql.connector
from database.connessione import creaConnessione
from models.evento_sicurezza import EventoSicurezza


# CREATE
def inserisciEvento(evento: EventoSicurezza, id_utente=None):
    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor()

        query = """
            INSERT INTO eventi (id_categoria, indirizzo_ip, descrizione, gravita, stato, data_evento, id_utente)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """

        valori = (
            evento.id_categoria,
            evento.indirizzo_ip,
            evento.descrizione,
            evento.gravita,
            evento.stato,
            evento.data_evento,
            id_utente,
        )

        cursore.execute(query, valori)
        connessione.commit()

        evento.id_evento = cursore.lastrowid
        return evento.id_evento

    except mysql.connector.Error as errore:
        if connessione is not None:
            connessione.rollback()
        print(f"Errore durante l'inserimento: {errore}")
        raise

    finally:
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


def inserisciEventi(eventi):
    """Salva l'intera importazione in una transazione."""
    connessione = None
    cursore = None
    try:
        connessione = creaConnessione()
        cursore = connessione.cursor()
        cursore.executemany(
            """INSERT INTO eventi
               (id_categoria, indirizzo_ip, descrizione, gravita, stato, data_evento)
               VALUES (%s, %s, %s, %s, %s, %s)""",
            [
                (e.id_categoria, e.indirizzo_ip, e.descrizione, e.gravita, e.stato, e.data_evento)
                for e in eventi
            ],
        )
        connessione.commit()
    except mysql.connector.Error:
        if connessione is not None:
            connessione.rollback()
        raise
    finally:
```

```
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


# READ - Tutti gli eventi
def visualizzaEventi(
    gravita=None,
    stato=None,
    ordine="recenti",
    testo=None,
    id_utente=None,
    amministratore=False,
    id_importazione=None,
):
    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor(dictionary=True)

        query = """
            SELECT e.id_evento, e.id_categoria, c.nome AS categoria,
                   e.indirizzo_ip, e.descrizione, e.gravita, e.stato, e.data_evento, e.id_utente, e.id_importazione, u.us
ername AS autore
            FROM eventi e
            JOIN categorie c ON e.id_categoria = c.id_categoria
            LEFT JOIN utenti u ON e.id_utente = u.id_utente
            WHERE 1=1
        """

        valori = []
        if not amministratore:
            query += " AND e.id_utente = %s"
            valori.append(id_utente)
        if id_importazione is not None:
            query += " AND e.id_importazione = %s"
            valori.append(id_importazione)

        if testo:
            if testo.isascii() and testo.isdecimal():
                cifre = testo.lstrip("0") or "0"
                if len(cifre) <= 20 and int(cifre) <= 18446744073709551615:
                    query += " AND e.id_evento = %s"
                    valori.append(int(cifre))
                else:
                    query += " AND 1=0"
            else:
                query += " AND (e.indirizzo_ip LIKE %s OR e.descrizione LIKE %s)"
                valori.extend([f"%{testo}%", f"%{testo}%"])

        if gravita:
            query += " AND e.gravita = %s"
            valori.append(gravita)

        if stato:
            query += " AND e.stato = %s"
            valori.append(stato)

        if ordine == "vecchi":
            query += " ORDER BY e.data_evento ASC, e.id_evento ASC"
        else:
            query += " ORDER BY e.data_evento DESC, e.id_evento DESC"

        cursore.execute(query, tuple(valori))
        return cursore.fetchall()

    except mysql.connector.Error as errore:
        print(f"Errore durante la lettura: {errore}")
        raise

    finally:
```

```
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


# READ - Singolo evento
def cercaEvento(id_evento: int, id_utente=None, amministratore=False):
    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor(dictionary=True)

        query = """
            SELECT e.id_evento, e.id_categoria, c.nome AS categoria,
                   e.indirizzo_ip, e.descrizione, e.gravita, e.stato, e.data_evento, e.id_utente, e.id_importazione, u.us
ername AS autore
            FROM eventi e
            JOIN categorie c ON e.id_categoria = c.id_categoria
            LEFT JOIN utenti u ON e.id_utente = u.id_utente
            WHERE e.id_evento = %s
        """

        valori = (id_evento,)
        if not amministratore:
            query += " AND e.id_utente = %s"
            valori += (id_utente,)
        cursore.execute(query, valori)
        return cursore.fetchone()

    except mysql.connector.Error as errore:
        print(f"Errore durante la ricerca: {errore}")
        raise

    finally:
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


# UPDATE
def modificaEvento(evento: EventoSicurezza, id_utente=None, amministratore=False):
    if evento.id_evento is None:
        raise ValueError("Impossibile modificare un evento senza ID.")

    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor()

        query = """
            UPDATE eventi
            SET id_categoria = %s, indirizzo_ip = %s, descrizione = %s,
                gravita = %s, stato = %s
            WHERE id_evento = %s
        """

        valori = (
            evento.id_categoria,
            evento.indirizzo_ip,
            evento.descrizione,
            evento.gravita,
            evento.stato,
            evento.id_evento,
        )

        if not amministratore:
            query += " AND id_utente = %s"
            valori += (id_utente,)
```

```
        cursore.execute(query, valori)
        righe_modificate = cursore.rowcount
        connessione.commit()

        return righe_modificate > 0

    except mysql.connector.Error as errore:
        if connessione is not None:
            connessione.rollback()
        print(f"Errore durante la modifica: {errore}")
        raise

    finally:
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


# DELETE
def eliminaEvento(id_evento: int, id_utente=None, amministratore=False):
    if type(id_evento) is not int or id_evento <= 0:
        raise ValueError("L'ID dell'evento deve essere un intero positivo.")

    connessione = None
    cursore = None

    try:
        connessione = creaConnessione()
        cursore = connessione.cursor()

        query = "DELETE FROM eventi WHERE id_evento = %s"

        valori = (id_evento,)
        if not amministratore:
            query += " AND id_utente = %s"
            valori += (id_utente,)
        cursore.execute(query, valori)
        righe_eliminate = cursore.rowcount
        connessione.commit()

        return righe_eliminate > 0

    except mysql.connector.Error as errore:
        if connessione is not None:
            connessione.rollback()
        print(f"Errore durante l'eliminazione: {errore}")
        raise

    finally:
        if cursore is not None:
            cursore.close()
        if connessione is not None and connessione.is_connected():
            connessione.close()


def statisticheEventi(id_utente=None, amministratore=False):
    connessione = creaConnessione()
    cursore = connessione.cursor(dictionary=True)

    try:
        query = """
            SELECT
                COUNT(*) AS totale,
                SUM(CASE WHEN gravita = 'Critica' THEN 1 ELSE 0 END) AS critici,
                SUM(CASE WHEN stato = 'Nuovo' THEN 1 ELSE 0 END) AS nuovi,
                SUM(CASE WHEN stato = 'Risolto' THEN 1 ELSE 0 END) AS risolti
            FROM eventi
        """

        valori = ()
        if not amministratore:
            query += " WHERE id_utente = %s"
            valori = (id_utente,)
```

```
        cursore.execute(query, valori)
        statistiche = cursore.fetchone()

        return statistiche

    finally:
        cursore.close()
        connessione.close()
```

database/importazioni_repository.py

76 righe

```
import mysql.connector
from database.strumenti import apriCursore


def salvaImportazione(eventi, id_utente, nome_file, contenuto, righe_valide, righe_errate, token):
    """Registra file e tutti gli eventi nella stessa transazione; token evita doppi invii."""
    try:
        with apriCursore() as (connessione, cursore):
            cursore.execute(
                '''INSERT INTO importazioni (id_utente, token_importazione, nome_file,
                contenuto, dimensione, righe_valide, righe_errate, eventi_importati)
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s)''',
                (
                    id_utente,
                    token,
                    nome_file,
                    contenuto,
                    len(contenuto),
                    righe_valide,
                    righe_errate,
                    len(eventi),
                ),
            )
            id_importazione = cursore.lastrowid
            cursore.executemany(
                '''INSERT INTO eventi (id_categoria, indirizzo_ip, descrizione,
                gravita, stato, data_evento, id_utente, id_importazione) VALUES (%s,%s,%s,%s,%s,%s,%s,%s)''',
                [
                    (
                        e.id_categoria,
                        e.indirizzo_ip,
                        e.descrizione,
                        e.gravita,
                        e.stato,
                        e.data_evento,
                        id_utente,
                        id_importazione,
                    )
                    for e in eventi
                ],
            )
            connessione.commit()
            return id_importazione
    except mysql.connector.IntegrityError as errore:
        if errore.errno == 1062:
            raise ValueError(
                'Questo file è già stato importato da questa anteprima. Consulta lo storico.'
            ) from None
        raise


def visualizzaImportazioni(id_utente, amministratore=False):
    with apriCursore(True) as (_, cursore):
        query = '''SELECT i.id_importazione, i.id_utente, i.nome_file, i.dimensione,
            i.righe_valide, i.righe_errate, i.eventi_importati, i.data_importazione,
            u.username, (SELECT COUNT(*) FROM eventi e WHERE e.id_importazione=i.id_importazione) AS eventi_attuali
            FROM importazioni i JOIN utenti u ON i.id_utente=u.id_utente'''
        if not amministratore:
            query += ' WHERE i.id_utente=%s'
        cursore.execute(
            query + ' ORDER BY i.data_importazione DESC, i.id_importazione DESC',
            () if amministratore else (id_utente,),
        )
```

```
        return cursore.fetchall()


def cercaImportazione(id_importazione, id_utente, amministratore=False):
    with apriCursore(True) as (_, cursore):
        query = '''SELECT i.*, u.username FROM importazioni i JOIN utenti u ON i.id_utente=u.id_utente
            WHERE i.id_importazione=%s'''
        valori = [id_importazione]
        if not amministratore:
            query += ' AND i.id_utente=%s'
            valori.append(id_utente)
        cursore.execute(query, tuple(valori))
        return cursore.fetchone()
```

database/migrazioni.py

50 righe

```
from database.strumenti import apriCursore


def preparaAccount():
    """Aggiunge tabelle e collegamenti, conservando gli eventi preesistenti."""
    with apriCursore() as (connessione, cursore):
        cursore.execute("""CREATE TABLE IF NOT EXISTS utenti (
            id_utente INT AUTO_INCREMENT PRIMARY KEY,
            username VARCHAR(40) NOT NULL UNIQUE,
            email VARCHAR(254) NOT NULL UNIQUE,
            password_hash VARCHAR(255) NOT NULL,
            ruolo ENUM('utente', 'admin') NOT NULL DEFAULT 'utente',
            attivo BOOLEAN NOT NULL DEFAULT TRUE,
            versione_sessione INT NOT NULL DEFAULT 0,
            tentativi_login INT NOT NULL DEFAULT 0,
            bloccato_fino DATETIME NULL,
            data_creazione DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
            ultimo_accesso DATETIME NULL
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4""")
        cursore.execute('SHOW COLUMNS FROM utenti')
        colonne_utenti = {c[0] for c in cursore.fetchall()}
        if 'password_da_cambiare' not in colonne_utenti:
            cursore.execute(
                'ALTER TABLE utenti ADD COLUMN password_da_cambiare BOOLEAN NOT NULL DEFAULT FALSE'
            )
        cursore.execute("""CREATE TABLE IF NOT EXISTS importazioni (
            id_importazione INT AUTO_INCREMENT PRIMARY KEY,
            id_utente INT NOT NULL,
            token_importazione VARCHAR(64) NOT NULL UNIQUE,
            nome_file VARCHAR(255) NOT NULL,
            contenuto MEDIUMBLOB NOT NULL,
            dimensione INT NOT NULL,
            righe_valide INT NOT NULL,
            righe_errate INT NOT NULL,
            eventi_importati INT NOT NULL,
            data_importazione DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (id_utente) REFERENCES utenti(id_utente),
            INDEX (id_utente, data_importazione)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4""")
        cursore.execute("SHOW COLUMNS FROM eventi")
        colonne = {c[0] for c in cursore.fetchall()}
        if 'id_utente' not in colonne:
            cursore.execute(
                "ALTER TABLE eventi ADD COLUMN id_utente INT NULL, ADD CONSTRAINT fk_eventi_utente FOREIGN KEY (id_utente
) REFERENCES utenti(id_utente), ADD INDEX idx_eventi_utente (id_utente)"
            )
        if 'id_importazione' not in colonne:
            cursore.execute(
                "ALTER TABLE eventi ADD COLUMN id_importazione INT NULL, ADD CONSTRAINT fk_eventi_importazione FOREIGN KE
Y (id_importazione) REFERENCES importazioni(id_importazione), ADD INDEX idx_eventi_importazione (id_importazione)"
            )
        connessione.commit()
```

database/strumenti.py

18 righe

```
from contextlib import contextmanager
from database.connessione import creaConnessione


@contextmanager
def apriCursore(dizionario=False):
    connessione = creaConnessione()
    cursore = None
    try:
        cursore = connessione.cursor(dictionary=dizionario)
        yield connessione, cursore
    except Exception:
        connessione.rollback()
        raise
    finally:
        if cursore is not None:
            cursore.close()
        connessione.close()
```

database/utenti_repository.py

169 righe

```
from datetime import datetime, timedelta
import mysql.connector
from werkzeug.security import generate_password_hash, check_password_hash
from database.strumenti import apriCursore
from services.account import validaAccount, validaPassword

# Un hash fittizio rende simile il controllo quando l'account non esiste.
HASH_FITTIZIO = generate_password_hash('account-inesistente-non-usare')


def cercaUtente(id_utente):
    with apriCursore(True) as (_, cursore):
        cursore.execute('SELECT * FROM utenti WHERE id_utente=%s', (id_utente,))
        return cursore.fetchone()


def creaUtente(
    username, email, password, ruolo='utente', assegna_precedenti=False, cambio_password=False
):
    validaPassword(password)
    username, email = validaAccount(username, email, password)
    if ruolo not in ('utente', 'admin'):
        raise ValueError('Ruolo non valido.')
    try:
        with apriCursore() as (connessione, cursore):
            cursore.execute(
                'INSERT INTO utenti (username, email, password_hash, ruolo, password_da_cambiare) VALUES (%s,%s,%s,%s,%s)
',
                (username, email, generate_password_hash(password), ruolo, cambio_password),
            )
            id_utente = cursore.lastrowid
            if assegna_precedenti and ruolo == 'admin':
                cursore.execute(
                    'UPDATE eventi SET id_utente=%s WHERE id_utente IS NULL', (id_utente,)
                )
            connessione.commit()
            return id_utente
    except mysql.connector.IntegrityError as errore:
        if errore.errno == 1062:
            raise ValueError('Nome utente o email già utilizzati.') from None
        raise


def autenticaUtente(identificativo, password):
    identificativo = (identificativo or '').strip()
    if len(identificativo) > 254 or not isinstance(password, str) or len(password) > 128:
        return None
    with apriCursore(True) as (connessione, cursore):
        cursore.execute(
            'SELECT * FROM utenti WHERE username=%s OR email=%s FOR UPDATE',
            (identificativo, identificativo.lower()),
```

```
        )
        utente = cursore.fetchone()
        corretta = check_password_hash(
            utente['password_hash'] if utente else HASH_FITTIZIO, password
        )
        adesso = datetime.now()
        if (
            not utente
            or not utente['attivo']
            or (utente['bloccato_fino'] and utente['bloccato_fino'] > adesso)
        ):
            return None
        if not corretta:
            tentativi = (utente['tentativi_login'] if utente['bloccato_fino'] is None else 0) + 1
            blocco = adesso + timedelta(minutes=15) if tentativi >= 5 else None
            cursore.execute(
                'UPDATE utenti SET tentativi_login=%s, bloccato_fino=%s WHERE id_utente=%s',
                (tentativi, blocco, utente['id_utente']),
            )
            connessione.commit()
            return None
        cursore.execute(
            'UPDATE utenti SET tentativi_login=0, bloccato_fino=NULL, ultimo_accesso=%s WHERE id_utente=%s',
            (adesso, utente['id_utente']),
        )
        connessione.commit()
        return utente


def aggiornaProfilo(id_utente, username, email):
    username, email = validaAccount(username, email)
    try:
        with apriCursore() as (connessione, cursore):
            cursore.execute(
                'UPDATE utenti SET username=%s, email=%s WHERE id_utente=%s',
                (username, email, id_utente),
            )
            connessione.commit()
    except mysql.connector.IntegrityError as errore:
        if errore.errno == 1062:
            raise ValueError('Nome utente o email già utilizzati.') from None
        raise


def cambiaPassword(id_utente, attuale, nuova):
    from services.account import validaPassword

    validaPassword(nuova)
    with apriCursore(True) as (connessione, cursore):
        cursore.execute(
            'SELECT password_hash FROM utenti WHERE id_utente=%s FOR UPDATE', (id_utente,)
        )
        utente = cursore.fetchone()
        if not utente or not check_password_hash(utente['password_hash'], attuale):
            raise ValueError('La password attuale non è corretta.')
        cursore.execute(
            'UPDATE utenti SET password_hash=%s, password_da_cambiare=0, versione_sessione=versione_sessione+1 WHERE id_u
tente=%s',
            (generate_password_hash(nuova), id_utente),
        )
        connessione.commit()


def reimpostaPassword(id_utente, amministratore, nuova):
    from services.account import validaPassword

    validaPassword(nuova)
    if id_utente == amministratore:
        raise ValueError('Per la tua password usa la pagina Il mio account.')
    with apriCursore(True) as (connessione, cursore):
        cursore.execute('SELECT id_utente FROM utenti WHERE id_utente=%s FOR UPDATE', (id_utente,))
        if not cursore.fetchone():
            raise ValueError('Account non trovato.')
        cursore.execute(
```

```
            '''UPDATE utenti SET password_hash=%s, password_da_cambiare=1,
            versione_sessione=versione_sessione+1, tentativi_login=0, bloccato_fino=NULL
            WHERE id_utente=%s''',
            (generate_password_hash(nuova), id_utente),
        )
        connessione.commit()


def visualizzaUtenti():
    with apriCursore(True) as (_, cursore):
        cursore.execute('''SELECT u.id_utente, u.username, u.email, u.ruolo, u.attivo,
            u.data_creazione, u.ultimo_accesso,
            (SELECT COUNT(*) FROM eventi e WHERE e.id_utente=u.id_utente) AS eventi,
            (SELECT COUNT(*) FROM importazioni i WHERE i.id_utente=u.id_utente) AS importazioni
            FROM utenti u ORDER BY u.data_creazione DESC, u.id_utente DESC''')
        return cursore.fetchall()


def gestisciUtente(id_utente, amministratore, ruolo, attivo):
    if ruolo not in ('utente', 'admin'):
        raise ValueError('Ruolo non valido.')
    with apriCursore(True) as (connessione, cursore):
        # Blocca gli amministratori nello stesso ordine per proteggere l'ultimo attivo.
        cursore.execute(
            "SELECT id_utente FROM utenti WHERE ruolo='admin' AND attivo=1 ORDER BY id_utente FOR UPDATE"
        )
        admin_attivi = {u['id_utente'] for u in cursore.fetchall()}
        cursore.execute('SELECT * FROM utenti WHERE id_utente=%s FOR UPDATE', (id_utente,))
        utente = cursore.fetchone()
        if not utente:
            raise ValueError('Account non trovato.')
        if id_utente == amministratore and (not attivo or ruolo != 'admin'):
            raise ValueError(
                'Non puoi disattivare il tuo account o rimuovere il tuo ruolo amministratore.'
            )
        if (
            id_utente in admin_attivi
            and len(admin_attivi) == 1
            and (not attivo or ruolo != 'admin')
        ):
            raise ValueError('Deve restare almeno un amministratore attivo.')
        cursore.execute(
            'UPDATE utenti SET ruolo=%s, attivo=%s, versione_sessione=versione_sessione+1, tentativi_login=0, bloccato_fi
no=NULL WHERE id_utente=%s',
            (ruolo, bool(attivo), id_utente),
        )
        connessione.commit()
```

esempi/README.md

35 righe

```
# Log di esempio

I file contengono eventi inventati per provare CS Monitor. Gli indirizzi IPv4 `192.0.2.x` e IPv6 `2001:db8::` sono usati
per gli esempi; non rappresentano dispositivi reali.

| File | Eventi validi | Contenuto |
|---|---:|---|
| `eventi.log` | 4 | Esempio breve per iniziare. |
| `accessi.log` | 200 | Accessi riusciti, password errate, account bloccati e accessi sospetti. |
| `firewall.log` | 200 | Connessioni consentite, traffico bloccato e scansioni delle porte. |
| `malware.log` | 200 | Scansioni antivirus, file sospetti, quarantena e minacce simulate. |
| `sistema.log` | 200 | Backup, aggiornamenti, errori dei servizi e modifiche alla configurazione. |
| `eventi_misti.log` | 500 | Eventi di tutti gli scenari e di tutte le gravità. |
| `eventi_ipv6.log` | 100 | Eventi con indirizzi IPv6. |
| `volume_1000_eventi.log` | 1000 | Il massimo numero di righe consentito, per provare un'importazione grande. |
| `righe_errate.log` | 20 | Include anche 6 righe intenzionalmente errate per verificare i messaggi di errore. |

## Come usarli

1. Accedi e apri **Importa log**.
2. Scegli uno dei file e premi **Analizza file**.
3. Controlla l'anteprima e seleziona gli eventi desiderati. Usa **Seleziona tutti** per un file della stessa categoria.
4. Scegli la **Categoria da applicare** e premi **Applica ai selezionati**. Puoi ancora modificare la categoria di singol
i eventi.
```

5. Conferma l'importazione e consulta la dashboard o lo storico dei log.

Tutti i file sono in UTF-8 e inferiori a 256 KB. Per iniziare con pochi eventi usa `eventi.log`. Per `firewall.log`, ad esempio, puoi assegnare **Firewall e traffico bloccato** a tutte le righe in un solo passaggio.

In `righe_errate.log`, le prime 20 righe sono valide. Le righe dalla 21 alla 26 provano rispettivamente: formato errato, IP non valido, data inesistente, ora non valida, livello non supportato e descrizione vuota. Il risultato atteso è **20 eventi validi e 6 errori**.

Il formato usato è:

```text
2026-10-13 08:00:00 | 192.0.2.1 | INFO | Accesso riuscito al portale aziendale
```

Livelli: `INFO`, `WARNING`, `ERROR`, `CRITICAL`. Le categorie si scelgono durante l'importazione. Caricare di nuovo un file già importato crea una nuova importazione e nuovi eventi.

models/categoria.py

28 righe

```
class Categoria:

    def __init__(self, id_categoria: int, nome: str):
        self.id_categoria = id_categoria
        self.nome = nome

    @property
    def id_categoria(self):
        return self._id_categoria

    @id_categoria.setter
    def id_categoria(self, valore):
        if type(valore) is not int or valore <= 0:
            raise ValueError("L'ID della categoria deve essere un intero positivo.")
        self._id_categoria = valore

    @property
    def nome(self):
        return self._nome

    @nome.setter
    def nome(self, valore):
        if not isinstance(valore, str) or not valore.strip():
            raise ValueError("Il nome della categoria non può essere vuoto.")
        self._nome = valore.strip()

    def __str__(self):
        return f"{self.id_categoria} - {self.nome}"
```

models/evento_sicurezza.py

101 righe

```
from datetime import datetime
from ipaddress import ip_address


class EventoSicurezza:

    GRAVITA_CONSENTITE = ("Bassa", "Media", "Alta", "Critica")
    STATI_CONSENTITI = ("Nuovo", "In analisi", "Risolto")

    def __init__(
        self,
        id_evento: int | None,
        id_categoria: int,
        indirizzo_ip: str,
        descrizione: str,
        gravita: str,
        stato: str = "Nuovo",
        data_evento: datetime | None = None,
    ):
        self.id_evento = id_evento
        self.id_categoria = id_categoria
        self.indirizzo_ip = indirizzo_ip
        self.descrizione = descrizione
```

```
        self.gravita = gravita
        self.stato = stato
        self.data_evento = data_evento if data_evento is not None else datetime.now()

    @property
    def id_evento(self):
        return self._id_evento

    @id_evento.setter
    def id_evento(self, valore):
        if valore is not None and (type(valore) is not int or valore <= 0):
            raise ValueError("L'ID dell'evento deve essere un intero positivo.")
        self._id_evento = valore

    @property
    def id_categoria(self):
        return self._id_categoria

    @id_categoria.setter
    def id_categoria(self, valore):
        if type(valore) is not int or valore <= 0:
            raise ValueError("L'ID della categoria deve essere un intero positivo.")
        self._id_categoria = valore

    @property
    def indirizzo_ip(self):
        return self._indirizzo_ip

    @indirizzo_ip.setter
    def indirizzo_ip(self, valore):
        try:
            ip_address(valore)
        except (ValueError, TypeError):
            raise ValueError("Indirizzo IP non valido.")
        self._indirizzo_ip = valore

    @property
    def descrizione(self):
        return self._descrizione

    @descrizione.setter
    def descrizione(self, valore):
        if not isinstance(valore, str) or not valore.strip():
            raise ValueError("La descrizione non può essere vuota.")
        self._descrizione = valore.strip()

    @property
    def gravita(self):
        return self._gravita

    @gravita.setter
    def gravita(self, valore):
        if valore not in self.GRAVITA_CONSENTITE:
            raise ValueError("Gravità non valida.")
        self._gravita = valore

    @property
    def stato(self):
        return self._stato

    @stato.setter
    def stato(self, valore):
        if valore not in self.STATI_CONSENTITI:
            raise ValueError("Stato non valido.")
        self._stato = valore

    @property
    def data_evento(self):
        return self._data_evento

    @data_evento.setter
    def data_evento(self, valore):
        if not isinstance(valore, datetime):
            raise ValueError("La data deve essere un oggetto datetime.")
```

```
        self._data_evento = valore

    def __str__(self):
        return f"Evento {self.id_evento} | Categoria: {self.id_categoria} | IP: {self.indirizzo_ip} | Gravità: {self.grav
ita} | Stato: {self.stato}"
```

routes/account.py

134 righe

```
from flask import Blueprint, render_template, request, redirect, url_for, session, g, flash
from database.utenti_repository import (
    creaUtente,
    autenticaUtente,
    aggiornaProfilo,
    cambiaPassword,
    visualizzaUtenti,
    gestisciUtente,
    reimpostaPassword,
)
from services.account import soloAdmin

account = Blueprint('account', __name__)


@account.route('/registrazione', methods=['GET', 'POST'])
def registrazione():
    errore = None
    if request.method == 'POST':
        try:
            if request.form.get('password') != request.form.get('conferma_password'):
                raise ValueError('Le password non coincidono.')
            # Il ruolo pubblico è sempre utente, indipendentemente dai dati inviati.
            creaUtente(
                request.form.get('username'),
                request.form.get('email'),
                request.form.get('password'),
            )
            flash('Registrazione completata. Ora puoi accedere.', 'success')
            return redirect(url_for('account.login'), code=303)
        except ValueError as e:
            errore = str(e)
    return render_template('registrazione.html', errore=errore)


@account.route('/login', methods=['GET', 'POST'])
def login():
    if g.utente:
        return redirect(url_for('dashboard'))
    errore = None
    if request.method == 'POST':
        utente = autenticaUtente(
            request.form.get('identificativo'), request.form.get('password', '')
        )
        if utente:
            session.clear()
            session.update(
                id_utente=utente['id_utente'], versione_sessione=utente['versione_sessione']
            )
            session.permanent = True
            return redirect(url_for('dashboard'), code=303)
        errore = 'Accesso non riuscito. Controlla i dati. Dopo 5 tentativi errati attendi 15 minuti o contatta un amminis
tratore.'
    return render_template('login.html', errore=errore)


@account.post('/logout')
def logout():
    session.clear()
    return redirect(url_for('account.login'), code=303)


@account.route('/account', methods=['GET', 'POST'])
def profilo():
    errore = None
    if request.method == 'POST':
```

```
        try:
            if request.form.get('azione') == 'password':
                if request.form.get('nuova_password') != request.form.get('conferma_password'):
                    raise ValueError('Le nuove password non coincidono.')
                cambiaPassword(
                    g.utente['id_utente'],
                    request.form.get('password_attuale', ''),
                    request.form.get('nuova_password'),
                )
                session.clear()
                flash(
                    'Password cambiata. Accedi nuovamente: tutte le vecchie sessioni sono state chiuse.',
                    'success',
                )
                return redirect(url_for('account.login'), code=303)
            aggiornaProfilo(
                g.utente['id_utente'], request.form.get('username'), request.form.get('email')
            )
            flash('Profilo aggiornato.', 'success')
            return redirect(url_for('account.profilo'), code=303)
        except ValueError as e:
            errore = str(e)
    return render_template('profilo.html', errore=errore)


@account.route('/amministrazione/utenti', methods=['GET', 'POST'])
@soloAdmin
def amministrazione():
    errore = None
    if request.method == 'POST':
        try:
            if request.form.get('azione') == 'crea':
                creaUtente(
                    request.form.get('username'),
                    request.form.get('email'),
                    request.form.get('password'),
                    request.form.get('ruolo', 'utente'),
                    cambio_password=True,
                )
                flash('Account creato.', 'success')
            elif request.form.get('azione') == 'password':
                id_utente = request.form.get('id_utente', type=int)
                if not id_utente:
                    raise ValueError('Account non valido.')
                reimpostaPassword(
                    id_utente, g.utente['id_utente'], request.form.get('nuova_password')
                )
                flash(
                    'Password temporanea impostata. L’utente dovrà cambiarla al prossimo accesso.',
                    'success',
                )
            else:
                id_utente = request.form.get('id_utente', type=int)
                if not id_utente:
                    raise ValueError('Account non valido.')
                gestisciUtente(
                    id_utente,
                    g.utente['id_utente'],
                    request.form.get('ruolo'),
                    request.form.get('attivo') == '1',
                )
                flash(
                    'Account aggiornato. Le sessioni precedenti e il blocco per tentativi errati sono stati azzerati.',
                    'success',
                )
            return redirect(url_for('account.amministrazione'), code=303)
        except ValueError as e:
            errore = str(e)
    return render_template('amministrazione.html', utenti=visualizzaUtenti(), errore=errore)
```

scripts/setup_account.py

63 righe

```
"""Migrazione esplicita: preserva gli eventi e crea il primo amministratore."""

import sys
import secrets
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from dotenv import load_dotenv

load_dotenv(ROOT / '.env')
from database.migrazioni import preparaAccount
from database.strumenti import apriCursore
from database.utenti_repository import creaUtente
from database.categorie_repository import aggiungiCategoriePredefinite


def main():
    import argparse
    import os

    parser = argparse.ArgumentParser(description='Prepara gli account di CS Monitor.')
    parser.add_argument('--username', default='admin')
    parser.add_argument('--email', required=True)
    args = parser.parse_args()
    preparaAccount()
    aggiunte = aggiungiCategoriePredefinite()
    print(f'Categorie di sicurezza aggiornate: {aggiunte} nuove categorie.')
    # Una chiave stabile evita di scollegare tutti a ogni riavvio.
    if not os.getenv('SECRET_KEY'):
        with (ROOT / '.env').open('a', encoding='utf-8') as file:
            file.write('\nSECRET_KEY=' + secrets.token_hex(32) + '\n')
    with apriCursore(True) as (_, cursore):
        cursore.execute("SELECT id_utente FROM utenti WHERE ruolo='admin' AND attivo=1")
        if cursore.fetchall():
            print(
                'Database aggiornato. Un amministratore attivo esiste già: nessuna credenziale modificata.'
            )
            return
    password = secrets.token_urlsafe(24)
    # Scrive le credenziali prima della creazione, senza sovrascrivere credenziali precedenti.
    destinazione = ROOT / 'instance' / 'credenziali-admin.txt'
    destinazione.parent.mkdir(exist_ok=True)
    with destinazione.open('x', encoding='utf-8') as file:
        file.write(
            f'CS Monitor - primo amministratore\nUsername: {args.username}\nEmail: {args.email}\nPassword: {password}\n\n
Cambia la password dopo il primo accesso dalla pagina del tuo account.\n'
        )
    creaUtente(
        args.username,
        args.email,
        password,
        ruolo='admin',
        assegna_precedenti=True,
        cambio_password=True,
    )
    print(
        'Database aggiornato. Amministratore creato. Credenziali in instance/credenziali-admin.txt (escluso da Git).'
    )


if __name__ == '__main__':
    main()
```

services/account.py

104 righe

```
import re
from functools import wraps
from datetime import timedelta
import secrets
from flask import g, session, request, abort, redirect, url_for
```

```
USERNAME = re.compile(r"^[A-Za-z0-9_.-]{3,40}$")
EMAIL = re.compile(r"^[^\s@]+@[^\s@]+\.[^\s@]+$")


def validaAccount(username, email, password=None):
    username = (username or '').strip()
    email = (email or '').strip().lower()
    if not USERNAME.fullmatch(username):
        raise ValueError(
            'Il nome utente deve contenere da 3 a 40 lettere, numeri, punti, trattini o underscore.'
        )
    if len(email) > 254 or not EMAIL.fullmatch(email):
        raise ValueError('Inserisci un indirizzo email valido.')
    if password is not None:
        validaPassword(password)
    return username, email


def validaPassword(password):
    if not isinstance(password, str) or not 10 <= len(password) <= 128:
        raise ValueError('La password deve contenere da 10 a 128 caratteri.')


def tokenCsrf():
    if 'csrf_token' not in session:
        session['csrf_token'] = secrets.token_urlsafe(32)
    return session['csrf_token']


def soloAdmin(funzione):
    @wraps(funzione)
    def protetta(*args, **kwargs):
        if g.utente is None or g.utente['ruolo'] != 'admin':
            abort(403)
        return funzione(*args, **kwargs)

    return protetta


def configuraSicurezza(app):
    from database import utenti_repository

    pubbliche = {'home', 'info', 'static', 'account.login', 'account.registrazione'}
    app.config.update(
        SESSION_COOKIE_HTTPONLY=True,
        SESSION_COOKIE_SAMESITE='Lax',
        PERMANENT_SESSION_LIFETIME=timedelta(hours=2),
    )

    @app.before_request
    def caricaSessione():
        g.utente = None
        id_utente = session.get('id_utente')
        if id_utente is not None:
            utente = utenti_repository.cercaUtente(id_utente)
            if (
                utente
                and utente['attivo']
                and utente['versione_sessione'] == session.get('versione_sessione')
            ):
                g.utente = utente
            else:
                session.clear()
        if request.endpoint is not None and request.endpoint not in pubbliche and g.utente is None:
            return redirect(url_for('account.login'))
        if request.method == 'POST':
            inviato = request.form.get('csrf_token', '')
            atteso = session.get('csrf_token', '')
            if not atteso or not secrets.compare_digest(
                inviato.encode('utf-8'), atteso.encode('utf-8')
            ):
                abort(
                    400,
```

```
                    description='Il modulo è scaduto o non valido. Ricarica la pagina e riprova.',
                )
        if (
            g.utente
            and g.utente.get('password_da_cambiare')
            and request.endpoint is not None
            and request.endpoint not in pubbliche | {'account.profilo', 'account.logout'}
        ):
            return redirect(url_for('account.profilo'))

    @app.context_processor
    def datiAccount():
        return {'utente_corrente': getattr(g, 'utente', None), 'csrf_token': tokenCsrf}

    @app.after_request
    def intestazioniSicure(risposta):
        risposta.headers['X-Content-Type-Options'] = 'nosniff'
        risposta.headers['X-Frame-Options'] = 'SAMEORIGIN'
        risposta.headers['Referrer-Policy'] = 'same-origin'
        if getattr(g, 'utente', None) is not None or request.endpoint in {
            'account.login',
            'account.registrazione',
        }:
            risposta.headers['Cache-Control'] = 'no-store'
        return risposta
```

services/log_importer.py

54 righe

```
import re
from datetime import datetime
from models.evento_sicurezza import EventoSicurezza

MAX_BYTES = 256 * 1024
MAX_RIGHE = 1000
GRAVITA = {"INFO": "Bassa", "WARNING": "Media", "ERROR": "Alta", "CRITICAL": "Critica"}
FORMATO = re.compile(
    r"^(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\s*\|\s*([^|]+?)\s*\|\s*([^|]+?)\s*\|\s*(.*)$"
)


def analizzaLog(contenuto):
    """Restituisce righe valide ed errori con i numeri di riga originali."""
    if len(contenuto) > MAX_BYTES:
        raise ValueError("Il file non può superare 256 KB.")
    try:
        righe = contenuto.decode("utf-8-sig").splitlines()
    except UnicodeDecodeError:
        raise ValueError("Il file deve avere codifica UTF-8.") from None
    if len(righe) > MAX_RIGHE:
        raise ValueError("Il file non può contenere più di 1000 righe.")
    eventi, errori = [], []
    for numero, riga in enumerate(righe, 1):
        if not riga.strip():
            continue
        try:
            campi = FORMATO.fullmatch(riga.strip())
            if campi is None:
                raise ValueError("Formato non valido: usa data | IP | livello | messaggio.")
            data_testo, ip, livello, descrizione = campi.groups()
            if livello not in GRAVITA:
                raise ValueError("Livello non valido: usa INFO, WARNING, ERROR o CRITICAL.")
            try:
                data = datetime.strptime(data_testo, "%Y-%m-%d %H:%M:%S")
            except ValueError:
                raise ValueError("Data o ora non valida.") from None
            evento = EventoSicurezza(
                None, 1, ip.strip(), descrizione, GRAVITA[livello], data_evento=data
            )
            eventi.append(
                {
                    "riga": numero,
                    "data_evento": data_testo,
                    "indirizzo_ip": evento.indirizzo_ip,
                    "descrizione": evento.descrizione,
```

```
                    "gravita": evento.gravita,
                }
            )
        except ValueError as errore:
            errori.append({"riga": numero, "messaggio": str(errore)})
    if not eventi and not errori:
        raise ValueError("Il file è vuoto o contiene soltanto righe vuote.")
    return eventi, errori
```

static/css/stile.css

316 righe

```
body {
    background-color: #f4f6f9;
    font-family: Arial, sans-serif;
}

header h1 {
    color: #004aad;
    font-weight: bold;
}

h2 {
    color: #1e293b;
    margin-bottom: 20px;
}

table {
    border-radius: 8px;
    overflow: hidden;
}

thead th {
    background-color: #004aad !important;
    color: white !important;
}

/* PAGINA ERRORE 404 */

.pagina-errore {
    background-color: #f4f6f9;
}

.codice-errore {
    font-size: 150px;
    font-weight: 800;
    color: #004aad;
    line-height: 1;
    margin-bottom: 20px;
}

.pagina-errore h2 {
    font-size: 32px;
    font-weight: 700;
    color: #1e293b;
}

.pagina-errore p {
    font-size: 17px;
    margin-top: 15px;
}

.pagina-errore .btn-applicaFiltri {
    background-color: #004aad;
    border-color: #004aad;
    padding: 12px 25px;
    border-radius: 8px;
}

.pagina-errore .btn-applicaFiltri:hover {
    background-color: #003580;
    border-color: #003580;
}

.btn-azioni {
```

```
    background-color: #004aad;
    color: white;
    border: none;
}

.btn-azioni:hover {
    background-color: #003080;
    color: white;
}

.btn-applicaFiltri {
    background-color: #004aad;
    color: white;
    border: none;
}

.btn-applicaFiltri:hover {
    background-color: #003580;
    color: white;
}

.home-icon {
    font-size: 35px;
    color: #004AAD;
}

.home-report-icon {
    font-size: 45px;
    color: #004AAD;
}

.footer-csm {
    background-color: #212529;
    color: #adb5bd;
    font-size: 13px;
}

.footer-csm strong {
    color: white;
}


.dettagli-card {
    width: 100%;
    max-width: 850px;
    border: 1px solid #dee2e6;
    border-radius: 15px;
    background-color: #ffffff;
}

.dettagli-card h2 {
    font-size: 28px;
    font-weight: 600;
}

.dettagli-card p {
    margin-top: 6px;
    margin-bottom: 0;
}

.dettagli-card .badge {
    font-size: 13px;
    padding: 10px 14px;
}

@media (max-width: 576px) {
    .codice-errore {
        font-size: 100px;
    }
}

/* SELETTORE FILE IN ITALIANO */
.selettore-file {
    position: relative;
```

```
    display: flex;
    align-items: center;
    overflow: hidden;
    border: 1px solid #dee2e6;
    border-radius: 6px;
    background-color: #fff;
}

.selettore-file:focus-within {
    border-color: #004aad;
    box-shadow: 0 0 0 3px rgba(0, 74, 173, 0.2);
}

.selettore-file-input {
    position: absolute;
    inset: 0;
    width: 100%;
    height: 100%;
    opacity: 0;
    cursor: pointer;
}

.selettore-file-pulsante {
    flex-shrink: 0;
    padding: 8px 12px;
    border-right: 1px solid #dee2e6;
    background-color: #eaf1fb;
    color: #004aad;
}

.selettore-file-nome {
    min-width: 0;
    padding: 8px 12px;
    overflow: hidden;
    white-space: nowrap;
    text-overflow: ellipsis;
}


.info-introduzione {
    max-width: 700px;
}

.info-card {
    border: 1px solid #dee2e6;
    border-radius: 15px;
    background-color: #fff;
    box-shadow: 0 4px 18px rgba(30, 41, 59, 0.04);
}

.info-icona {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 52px;
    height: 52px;
    flex-shrink: 0;
    border-radius: 14px;
    background-color: #eaf1fb;
    color: #004aad;
    font-size: 26px;
}

.info-testo {
    line-height: 1.8;
}

.info-funzionalita li {
    display: flex;
    align-items: baseline;
    gap: 12px;
    padding: 8px 0;
    color: #1e293b;
}
```

```
.info-funzionalita i {
    color: #004aad;
    flex-shrink: 0;
}

.info-tecnologia {
    padding: 8px 14px;
    border: 1px solid #d7e4f7;
    border-radius: 8px;
    background-color: #eaf1fb;
    color: #004aad;
    font-size: 14px;
    font-weight: 600;
}

.info-profilo {
    border-top: 4px solid #004aad;
}

.info-avatar {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 76px;
    height: 76px;
    border-radius: 50%;
    object-fit: cover;
    background-color: #004aad;
    color: #fff;
    font-size: 26px;
    font-weight: 700;
}

.info-contatti {
    border-top: 1px solid #dee2e6;
}

.info-contatto {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 12px 16px;
    border: 1px solid #dee2e6;
    border-radius: 8px;
    color: #1e293b;
    text-decoration: none;
    transition: background-color 0.15s, border-color 0.15s;
}

.info-contatto i {
    color: #004aad;
}

.info-contatto:hover,
.info-contatto:focus-visible {
    background-color: #eaf1fb;
    border-color: #004aad;
    color: #004aad;
}

/* Account e storico: stessi colori e schede del resto del progetto. */
.account-card {
    max-width: 560px;
}


/* Colori comuni anche per collegamenti e stati dei moduli. */
a {
    color: #004aad;
}

a:hover {
    color: #003580;
```

```
}

.btn-applicaFiltri:focus-visible,
.btn-azioni:focus-visible {
    background-color: #003580;
    color: #fff;
    box-shadow: 0 0 0 3px rgba(0, 74, 173, 0.25);
}

.btn-applicaFiltri:active,
.btn-azioni:active,
.btn-applicaFiltri:disabled,
.btn-azioni:disabled {
    background-color: #003580;
    color: #fff;
}

.form-control:focus,
.form-select:focus {
    border-color: #004aad;
    box-shadow: 0 0 0 3px rgba(0, 74, 173, 0.2);
}

.navbar .nav-link {
    color: rgba(255, 255, 255, 0.65);
}

.navbar .nav-link:hover,
.navbar .nav-link:focus-visible {
    color: #fff;
}
```

static/img/logo.png

binario

[contenuto omesso: file binario]

static/img/profilo.png

binario

[contenuto omesso: file binario]

static/js/importa_log.js

62 righe

```
const fileLog = document.getElementById('file_log');
const nomeFileLog = document.getElementById('nome-file-log');
fileLog.addEventListener('change', function() {
    nomeFileLog.textContent = fileLog.files.length ?
        fileLog.files[0].name :
        'Nessun file selezionato';
});

const selezioniEventi = Array.from(document.querySelectorAll('.selezione-evento'));
const categoriaGruppo = document.getElementById('categoria-gruppo');
const applicaCategoria = document.getElementById('applica-categoria');
const conteggioSelezionati = document.getElementById('conteggio-selezionati');
const esitoCategoria = document.getElementById('esito-categoria');

function aggiornaSelezione() {
    let selezionati = 0;
    selezioniEventi.forEach(function(checkbox) {
        const categoria = checkbox.closest('tr').querySelector('select');
        categoria.disabled = !checkbox.checked;
        categoria.required = checkbox.checked;
        if (checkbox.checked) selezionati++;
    });
    if (conteggioSelezionati) {
        conteggioSelezionati.textContent = `${selezionati} eventi selezionati su ${selezioniEventi.length}.`;
        applicaCategoria.disabled = !categoriaGruppo.value || selezionati === 0;
    }
    if (esitoCategoria) esitoCategoria.textContent = '';
}

selezioniEventi.forEach(function(checkbox) {
    checkbox.addEventListener('change', aggiornaSelezione);
});
```

```
// I controlli di gruppo esistono soltanto dopo aver caricato un file valido.
if (categoriaGruppo) {
    categoriaGruppo.addEventListener('change', aggiornaSelezione);
    document.getElementById('seleziona-tutti').addEventListener('click', function() {
        selezioniEventi.forEach(function(checkbox) {
            checkbox.checked = true;
        });
        aggiornaSelezione();
    });
    document.getElementById('deseleziona-tutti').addEventListener('click', function() {
        selezioniEventi.forEach(function(checkbox) {
            checkbox.checked = false;
        });
        aggiornaSelezione();
    });
    applicaCategoria.addEventListener('click', function() {
        if (!categoriaGruppo.value) return;
        let aggiornati = 0;
        selezioniEventi.forEach(function(checkbox) {
            if (checkbox.checked) {
                checkbox.closest('tr').querySelector('select').value = categoriaGruppo.value;
                aggiornati++;
            }
        });
        const nome = categoriaGruppo.options[categoriaGruppo.selectedIndex].text;
        esitoCategoria.textContent = `Categoria “${nome}” assegnata a ${aggiornati} eventi. Puoi ancora modificarla sulle
 singole righe.`;
    });
}
aggiornaSelezione();
```

templates/404.html

23 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>404 - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="pagina-errore">
        <main class="container d-flex justify-content-center align-items-center min-vh-100">
            <div class="text-center">
                <h1 class="codice-errore">404</h1>
                <h2>Pagina non trovata</h2>
                <p class="text-secondary">La pagina o l'evento che stai cercando non esiste.</p>
                <a href="{{ url_for('dashboard') }}"
                   class="btn btn-applicaFiltri mt-3">Torna alla dashboard</a>
            </div>
        </main>
    </body>
</html>
```

templates/amministrazione.html

149 righe

```
{% extends "base_account.html" %}
{% block titolo %}Gestione account{% endblock %}
{% block contenuto %}
    <h1>Gestione account</h1>
    <p class="text-secondary">Crea account, assegna i ruoli e gestisci l'accesso. La disattivazione conserva eventi e imp
ortazioni.</p>
    <div class="card info-card mb-4">
        <div class="card-body p-4">
            <h2 class="h4">Account registrati</h2>
            <div class="table-responsive">
                <table class="table align-middle">
                    <thead>
                        <tr>
                            <th>Utente</th>
```

```
                            <th>Attività</th>
                            <th>Ultimo accesso</th>
                            <th>Gestione accesso</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for utente in utenti %}
                            <tr>
                                <td>
                                    <strong>{{ utente.username }}</strong>
                                    <br>
                                    <span class="text-secondary">{{ utente.email }}</span>
                                    <br>
                                    <span class="badge text-bg-{{ 'success' if utente.attivo else 'secondary' }}">{{ 'Att
ivo' if utente.attivo else 'Disattivato' }}</span>
                                </td>
                                <td>
                                    {{ utente.eventi }} eventi
                                    <br>
                                    {{ utente.importazioni }} importazioni
                                </td>
                                <td>{{ utente.ultimo_accesso or 'Mai' }}</td>
                                <td>
                                    <form method="post" class="d-flex flex-wrap gap-2">
                                        <input type="hidden"
                                               name="csrf_token"
                                               value="{{ csrf_token() }}">
                                        <input type="hidden"
                                               name="id_utente"
                                               value="{{ utente.id_utente }}">
                                        <input type="hidden" name="azione" value="gestisci">
                                        <select class="form-select w-auto"
                                                name="ruolo"
                                                aria-label="Ruolo di {{ utente.username }}">
                                            {% for ruolo in ['utente', 'admin'] %}
                                                <option value="{{ ruolo }}" {% if utente.ruolo == ruolo %}selected{% endi
f %}>
                                                    {{ 'Amministratore' if ruolo == 'admin' else 'Utente' }}
                                                </option>
                                            {% endfor %}
                                        </select>
                                        <select class="form-select w-auto"
                                                name="attivo"
                                                aria-label="Stato di {{ utente.username }}">
                                            <option value="1" {% if utente.attivo %}selected{% endif %}>
                                                Attivo
                                            </option>
                                            <option value="0" {% if not utente.attivo %}selected{% endif %}>
                                                Disattivato
                                            </option>
                                        </select>
                                        <button class="btn btn-azioni"
                                                type="submit">Aggiorna</button>
                                    </form>
                                    {% if utente.id_utente != utente_corrente.id_utente %}
                                        <details class="mt-2">
                                            <summary>Reimposta password</summary>
                                            <form method="post" class="d-flex gap-2 mt-2">
                                                <input type="hidden"
                                                       name="csrf_token"
                                                       value="{{ csrf_token() }}">
                                                <input type="hidden"
                                                       name="id_utente"
                                                       value="{{ utente.id_utente }}">
                                                <input type="hidden" name="azione" value="password">
                                                <input class="form-control"
                                                       name="nuova_password"
                                                       type="password"
                                                       autocomplete="new-password"
                                                       minlength="10"
                                                       maxlength="128"
                                                       aria-label="Password temporanea per {{ utente.username }}"
                                                       placeholder="Password temporanea"
                                                       required>
```

```
                                            <button class="btn btn-applicaFiltri"
                                                    type="submit">Reimposta</button>
                                        </form>
                                        <p class="small text-secondary">Comunica la password temporanea all'utente. D
ovrà cambiarla al prossimo accesso.</p>
                                    </details>
                                {% endif %}
                            </td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
        <p class="small text-secondary mb-0">Ogni aggiornamento chiude le sessioni dell'account e azzera il blocco de
i tentativi di accesso. Il tuo account amministratore deve rimanere attivo.</p>
    </div>
</div>
<div class="card info-card">
    <div class="card-body p-4">
        <h2 class="h4">Crea un account</h2>
        <form method="post" class="row g-3">
            <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
            <input type="hidden" name="azione" value="crea">
            <div class="col-md-6">
                <label class="form-label" for="username">Nome utente</label>
                <input class="form-control"
                       id="username"
                       name="username"
                       minlength="3"
                       maxlength="40"
                       required>
            </div>
            <div class="col-md-6">
                <label class="form-label" for="email">Email</label>
                <input class="form-control"
                       id="email"
                       name="email"
                       type="email"
                       maxlength="254"
                       required>
            </div>
            <div class="col-md-6">
                <label class="form-label" for="password">Password iniziale</label>
                <input class="form-control"
                       id="password"
                       name="password"
                       type="password"
                       autocomplete="new-password"
                       minlength="10"
                       maxlength="128"
                       required>
            </div>
            <div class="col-md-6">
                <label class="form-label" for="ruolo">Ruolo</label>
                <select class="form-select" id="ruolo" name="ruolo">
                    <option value="utente">Utente</option>
                    <option value="admin">Amministratore</option>
                </select>
            </div>
            <div>
                <button class="btn btn-applicaFiltri" type="submit">Crea account</button>
            </div>
        </form>
    </div>
</div>
{% endblock %}
```

templates/base_account.html

28 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
        <title>{% block titolo %}Account{% endblock %} - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            {% with messaggi = get_flashed_messages(with_categories=true) %}
                {% for categoria, messaggio in messaggi %}
                    <div class="alert alert-{{ categoria }}" role="status">{{ messaggio }}</div>
                {% endfor %}
            {% endwith %}
            {% if errore %}
                <div class="alert alert-danger" role="alert">{{ errore }}</div>
            {% endif %}
            {% block contenuto %}{% endblock %}
        </main>
        {% include "footer.html" %}
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
    </body>
</html>
```

templates/dettagli_evento.html

83 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Dettagli evento - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.min.css">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container d-flex justify-content-center py-5 flex-grow-1">
            <div class="card dettagli-card shadow-sm">
                <div class="card-body p-4 p-md-5">
                    <div class="d-flex justify-content-between align-items-center mb-4">
                        <div>
                            <small class="text-secondary">EVENTO DI SICUREZZA</small>
                            <h2 class="mt-2 mb-0">Dettagli evento ID: {{ evento.id_evento }}</h2>
                        </div>
                        <span class="badge bg-danger">{{ evento.gravita }}</span>
                    </div>
                    <hr>
                    {% if utente_corrente.ruolo == "admin" and evento.autore %}
                        <p>Utente: <strong>{{ evento.autore }}</strong></p>
                    {% endif %}
                    {% if evento.id_importazione %}
                        <p>Origine: <a href="{{ url_for('dettagliImportazione', id_importazione=evento.id_importazione) }
}">File di log importato</a></p>
                    {% endif %}
                    <div class="row g-4 my-2">
                        <div class="col-md-6">
                            <small class="text-secondary">Categoria</small>
                            <p class="fw-semibold">{{ evento.categoria }}</p>
                        </div>
                        <div class="col-md-6">
                            <small class="text-secondary">Indirizzo IP</small>
                            <p class="fw-semibold">{{ evento.indirizzo_ip }}</p>
                        </div>
                        <div class="col-md-6">
                            <small class="text-secondary">Gravità</small>
                            <p class="fw-semibold">{{ evento.gravita }}</p>
                        </div>
                        <div class="col-md-6">
                            <small class="text-secondary">Stato</small>
```

```
                        <p class="fw-semibold">{{ evento.stato }}</p>
                    </div>
                    <div class="col-12">
                        <small class="text-secondary">Data e ora</small>
                        <p class="fw-semibold">{{ evento.data_evento }}</p>
                    </div>
                </div>
                <div class="mb-4">
                    <small class="text-secondary">Descrizione</small>
                    <div class="bg-light rounded p-3 mt-2">{{ evento.descrizione }}</div>
                </div>
                <hr>
                <div class="d-flex flex-wrap gap-2 mt-4">
                    <a href="{{ url_for('modificaEventoPagina', id_evento=evento.id_evento) }}"
                       class="btn btn-warning">
                        <i class="bi bi-pencil-square"></i> Modifica evento
                    </a>
                    <form action="{{ url_for('eliminaEventoPagina', id_evento=evento.id_evento) }}"
                          method="post"
                          onsubmit="return confirm('Sei sicuro di voler eliminare questo evento?');">
                        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                        <button type="submit" class="btn btn-danger">
                            <i class="bi bi-trash"></i> Elimina evento
                        </button>
                    </form>
                    <a href="{{ url_for('dashboard') }}" class="btn btn-applicaFiltri">
                        <i class="bi bi-arrow-left"></i> Torna alla dashboard
                    </a>
                </div>
            </div>
        </div>
    </main>
    {% include "footer.html" %}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

templates/dettagli_importazione.html

60 righe

```
{% extends "base_account.html" %}
{% block titolo %}Dettaglio importazione{% endblock %}
{% block contenuto %}
    <a class="btn btn-secondary"
       href="{{ url_for('storicoImportazioni') }}">← Torna allo storico</a>
    <h1 class="mt-3 text-break">{{ importazione.nome_file }}</h1>
    <p class="text-secondary">Importato da {{ importazione.username }} il {{ importazione.data_importazione }} · {{ impor
tazione.dimensione }} byte</p>
    <div class="card info-card mb-4">
        <div class="card-body p-4">
            <div class="row g-3 mb-4">
                <div class="col-sm-4">
                    <strong>{{ importazione.eventi_importati }}</strong> eventi importati
                </div>
                <div class="col-sm-4">
                    <strong>{{ importazione.righe_valide }}</strong> righe valide nel file
                </div>
                <div class="col-sm-4">
                    <strong>{{ importazione.righe_errate }}</strong> righe errate
                </div>
            </div>
            <a class="btn btn-applicaFiltri"
               href="{{ url_for('scaricaLog', id_importazione=importazione.id_importazione) }}">Scarica il file originale
</a>
            <p class="small text-secondary mt-3 mb-0">Il file originale resta disponibile anche se modifichi o elimini gl
i eventi. I conteggi descrivono l'importazione iniziale.</p>
        </div>
    </div>
    <h2 class="h4">Eventi attualmente presenti ({{ eventi|length }})</h2>
    <div class="table-responsive">
        <table class="table table-striped">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Categoria</th>
```

```
                    <th>IP</th>
                    <th>Descrizione</th>
                    <th>Stato</th>
                    <th></th>
                </tr>
            </thead>
            <tbody>
                {% for e in eventi %}
                    <tr>
                        <td>{{ e.id_evento }}</td>
                        <td>{{ e.categoria }}</td>
                        <td>{{ e.indirizzo_ip }}</td>
                        <td>{{ e.descrizione }}</td>
                        <td>{{ e.stato }}</td>
                        <td>
                            <a class="btn btn-azioni btn-sm"
                               href="{{ url_for('dettagliEvento', id_evento=e.id_evento) }}">Dettagli</a>
                        </td>
                    </tr>
                {% else %}
                    <tr>
                        <td colspan="6">Gli eventi di questa importazione sono stati eliminati.</td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
{% endblock %}
```

templates/errore_account.html

3 righe

```
{% extends "base_account.html" %}
{% block titolo %}Richiesta non consentita{% endblock %}
{% block contenuto %}<h1>Richiesta non consentita</h1><p>{{ messaggio }}</p><a class="btn btn-applicaFiltri" href="{{ url
_for('home') }}">Torna alla home</a>{% endblock %}
```

templates/errore_database.html

27 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Servizio temporaneamente non disponibile - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <div class="card dettagli-card mx-auto shadow-sm">
                <div class="card-body p-4 p-md-5 text-center">
                    <h1 class="h3 mb-3">Servizio temporaneamente non disponibile</h1>
                    <p class="text-secondary">Non è stato possibile completare l'operazione sul database. Riprova tra poc
o.</p>
                    <a href="{{ url_for('dashboard') }}"
                       class="btn btn-applicaFiltri">Riprova dalla dashboard</a>
                </div>
            </div>
        </main>
        {% include "footer.html" %}
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
    </body>
</html>
```

templates/eventi.html

161 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
        <title>CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container mt-5 flex-grow-1">
            {% for messaggio in get_flashed_messages(category_filter=['success']) %}
                <div class="alert alert-success" role="status">{{ messaggio }}</div>
            {% endfor %}
            <h2 class="mb-4">Statistiche eventi</h2>
            <p class="text-secondary">{{ 'Vista amministratore: tutti gli eventi degli utenti.' if utente_corrente.ruolo
== 'admin' else 'Il tuo spazio personale: eventi e statistiche del tuo account.' }}</p>
            <section class="container my-3">
                <div class="row g-2">
                    <div class="col-6 col-md-3">
                        <div class="card text-center shadow-sm">
                            <div class="card-body p-2">
                                <small>Eventi totali</small>
                                <h5 class="mb-0">{{ statistiche.totale }}</h5>
                            </div>
                        </div>
                    </div>
                    <div class="col-6 col-md-3">
                        <div class="card text-center shadow-sm">
                            <div class="card-body p-2">
                                <small>Eventi critici</small>
                                <h5 class="text-danger mb-0">{{ statistiche.critici or 0 }}</h5>
                            </div>
                        </div>
                    </div>
                    <div class="col-6 col-md-3">
                        <div class="card text-center shadow-sm">
                            <div class="card-body p-2">
                                <small>Eventi nuovi</small>
                                <h5 class="text-warning mb-0">{{ statistiche.nuovi or 0 }}</h5>
                            </div>
                        </div>
                    </div>
                    <div class="col-6 col-md-3">
                        <div class="card text-center shadow-sm">
                            <div class="card-body p-2">
                                <small>Eventi risolti</small>
                                <h5 class="text-success mb-0">{{ statistiche.risolti or 0 }}</h5>
                            </div>
                        </div>
                    </div>
                </div>
            </section>
            <section class="mb-4">
                <h2>Filtra eventi</h2>
                <form action="{{ url_for('dashboard') }}" method="get">
                    <div class="row g-3">
                        <!-- FILTRO GRAVITÀ -->
                        <div class="col-md-4">
                            <label for="gravita" class="form-label">Gravità</label>
                            <select class="form-select" id="gravita" name="gravita">
                                <option value="">Tutte le gravità</option>
                                {% for valore in ["Bassa", "Media", "Alta", "Critica"] %}
                                    <option value="{{ valore }}" {% if gravita == valore %}selected{% endif %}>
                                        {{ valore }}
                                    </option>
                                {% endfor %}
                            </select>
                        </div>
                        <!-- FILTRO STATO -->
                        <div class="col-md-4">
                            <label for="stato" class="form-label">Stato</label>
                            <select class="form-select" id="stato" name="stato">
                                <option value="">Tutti gli stati</option>
                                {% for valore in ["Nuovo", "In analisi", "Risolto"] %}
                                    <option value="{{ valore }}" {% if stato == valore %}selected{% endif %}>
```

```
                                    {{ valore }}
                                </option>
                            {% endfor %}
                        </select>
                    </div>
                    <!-- ORDINAMENTO -->
                    <div class="col-md-4">
                        <label for="ordine" class="form-label">Ordina per data e ora</label>
                        <select class="form-select" id="ordine" name="ordine">
                            <option value="recenti" {% if ordine == "recenti" %}selected{% endif %}>
                                Dal più recente
                            </option>
                            <option value="vecchi" {% if ordine == "vecchi" %}selected{% endif %}>
                                Dal più vecchio
                            </option>
                        </select>
                    </div>
                </div>
                <div class="mb-3 mt-2">
                    <input type="text"
                           name="testo"
                           class="form-control"
                           placeholder="Cerca per ID, IP o descrizione..."
                           value="{{ testo_ricerca or '' }}">
                </div>
                <div class="mt-3 d-flex align-items-center gap-2">
                    <button type="submit" class="btn btn-applicaFiltri">Applica filtri</button>
                    <a href="{{ url_for('dashboard') }}"
                       class="btn btn-secondary">Reimposta filtri</a>
                    <a href="{{ url_for('esportaEventi', gravita=gravita or '', stato=stato or '', ordine=ordine, tes
to=testo_ricerca or '') }}"
                       class="btn btn-success">
                        <i class="bi bi-download"></i> Esporta CSV
                    </a>
                </div>
            </form>
        </section>
        <h2>Eventi di sicurezza</h2>
        <div class="table-responsive">
            <table class="table table-striped table-hover">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Categoria</th>
                        <th>Indirizzo IP</th>
                        <th>Gravità</th>
                        <th>Stato</th>
                        <th>Data</th>
                        <th>Azioni</th>
                    </tr>
                </thead>
                <tbody>
                    {% for evento in eventi %}
                        <tr>
                            <td>
                                {{ evento.id_evento }}
                                {% if utente_corrente.ruolo == "admin" and evento.autore %}
                                    <br>
                                    <small class="text-secondary">{{ evento.autore }}</small>
                                {% endif %}
                            </td>
                            <td>{{ evento.categoria }}</td>
                            <td>{{ evento.indirizzo_ip }}</td>
                            <td>{{ evento.gravita }}</td>
                            <td>{{ evento.stato }}</td>
                            <td>{{ evento.data_evento }}</td>
                            <td>
                                <a href="{{ url_for('dettagliEvento', id_evento=evento.id_evento) }}"
                                   class="btn btn-azioni btn-sm">Dettagli</a>
                            </td>
                        </tr>
                    {% else %}
                        <tr>
                            <td colspan="7" class="text-center">Nessun evento presente.</td>
```

```
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
    </main>
    {% include "footer.html" %}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

templates/footer.html

10 righe

```
<footer class="footer-csm text-center py-3">
    <div class="container">
        <p class="mb-1">
            <strong>CS Monitor</strong> | Security Event Management
        </p>
        <p class="mb-0">
            &copy; 2026 CS Monitor - Progetto sviluppato con Python, Flask e MySQL - Filannino Marco
        </p>
    </div>
</footer>
```

templates/home.html

71 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Home - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.min.css">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <section class="home-hero text-center mb-5">
                <h1 class="display-5 fw-bold mt-3">CS Monitor</h1>
                <p class="lead text-secondary mx-auto home-descrizione">
                    Una piattaforma web per registrare, analizzare e gestire
                    gli eventi di sicurezza informatica in modo semplice e organizzato.
                </p>
                {% if not utente_corrente %}
                    <p class="text-secondary">Crea il tuo account per conservare eventi e file di log nel tuo spazio pers
onale.</p>
                    <a class="btn btn-applicaFiltri me-2"
                       href="{{ url_for('account.registrazione') }}">Crea un account</a><a class="btn btn-azioni" href="{
{ url_for('account.login') }}">Accedi</a>
                {% endif %}
            </section>
            <section class="row g-4 mb-5">
                <div class="col-md-4">
                    <div class="card home-card h-100">
                        <div class="card-body p-4">
                            <i class="bi bi-shield-plus home-icon"></i>
                            <h4 class="mt-3">Registra</h4>
                            <p class="text-secondary mb-0">
                                Registra gli eventi di sicurezza specificando
                                categoria, indirizzo IP, descrizione e gravità.
                            </p>
                        </div>
                    </div>
                </div>
                <div class="col-md-4">
                    <div class="card home-card h-100">
                        <div class="card-body p-4">
                            <i class="bi bi-search home-icon"></i>
                            <h4 class="mt-3">Analizza</h4>
```

```
                        <p class="text-secondary mb-0">
                            Consulta gli eventi, effettua ricerche avanzate
                            e applica filtri per individuare le criticità.
                        </p>
                    </div>
                </div>
            </div>
            <div class="col-md-4">
                <div class="card home-card h-100">
                    <div class="card-body p-4">
                        <i class="bi bi-clipboard-check home-icon"></i>
                        <h4 class="mt-3">Gestisci</h4>
                        <p class="text-secondary mb-0">
                            Modifica le informazioni degli eventi,
                            aggiorna lo stato di risoluzione e gestisci le segnalazioni.
                        </p>
                    </div>
                </div>
            </div>
        </section>
    </main>
    {% include "footer.html" %}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

templates/importa_log.html

172 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Importa log - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <h1>Importa log</h1>
            <p>Carica un file, controlla l'anteprima e scegli quali eventi salvare.</p>
            {% if errore %}
                <div class="alert alert-danger" role="alert">{{ errore }}</div>
            {% endif %}
            {% if messaggio %}
                <div class="alert alert-success" role="status">
                    {{ messaggio }} <a href="{{ url_for('dashboard') }}">Vai alla dashboard</a>
                </div>
            {% endif %}
            <div class="card mb-4">
                <div class="card-body">
                    <h2 class="h5">Formato del file</h2>
                    <pre class="text-wrap">2026-10-12 09:15:32 | 192.168.1.10 | WARNING | Login fallito per utente admin<
/pre>
                    <p>Livelli: INFO → Bassa, WARNING → Media, ERROR → Alta, CRITICAL → Critica.</p>
                    <p>File .log o .txt in UTF-8, massimo 256 KB e 1000 righe. Le righe vuote vengono ignorate.</p>
                    <form method="post"
                          enctype="multipart/form-data"
                          action="{{ url_for('importaLog') }}">
                        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                        <div class="selettore-file mb-3">
                            <input class="selettore-file-input"
                                   type="file"
                                   id="file_log"
                                   name="file_log"
                                   accept=".log,.txt"
                                   aria-label="Scegli file di log"
                                   aria-describedby="nome-file-log"
                                   required>
                            <span class="selettore-file-pulsante"
                                  aria-hidden="true">Scegli file</span>
```

```
                        <span class="selettore-file-nome"
                              id="nome-file-log"
                              aria-live="polite">Nessun file selezionato</span>
                    </div>
                    <button class="btn btn-applicaFiltri" type="submit">Analizza file</button>
                </form>
            </div>
        </div>
        {% if errori %}
            <div class="alert alert-warning">
                <h2 class="h5">Righe non importabili ({{ errori|length }})</h2>
                <ul class="mb-0">
                    {% for problema in errori %}
                        <li>Riga {{ problema.riga }}: {{ problema.messaggio }}</li>
                    {% endfor %}
                </ul>
            </div>
        {% endif %}
        {% if eventi and token %}
            <h2 class="h4">Anteprima: {{ eventi|length }} eventi validi</h2>
            <p>Assegna una categoria agli eventi selezionati, poi modifica le singole righe se necessario.</p>
            <form method="post" action="{{ url_for('importaLog') }}">
                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                <input type="hidden" name="azione" value="conferma">
                <input type="hidden" name="anteprima" value="{{ token }}">
                {% if not categorie %}
                    <p class="text-danger">Nessuna categoria disponibile nel database.</p>
                {% endif %}
                <div class="card info-card mb-3">
                    <div class="card-body">
                        <h3 class="h5">Categoria per più eventi</h3>
                        <div class="d-flex flex-wrap align-items-end gap-3">
                            <div class="flex-grow-1">
                                <label class="form-label"
                                       for="categoria-gruppo">Categoria da applicare</label>
                                <select class="form-select"
                                        id="categoria-gruppo"
                                        {% if not categorie %}disabled{% endif %}>
                                    <option value="">Seleziona una categoria</option>
                                    {% for categoria in categorie %}
                                        <option value="{{ categoria.id_categoria }}">{{ categoria.nome }}</option>
                                    {% endfor %}
                                </select>
                            </div>
                            <button class="btn btn-applicaFiltri"
                                    type="button"
                                    id="applica-categoria"
                                    disabled>Applica ai selezionati</button>
                            <button class="btn btn-azioni"
                                    type="button"
                                    id="seleziona-tutti">Seleziona tutti</button>
                            <button class="btn btn-secondary"
                                    type="button"
                                    id="deseleziona-tutti">Deseleziona tutti</button>
                        </div>
                        <p class="small text-secondary mt-3 mb-1">La categoria scelta sostituisce quella degli eventi
selezionati. Le altre righe restano invariate.</p>
                        <p class="mb-0"
                           id="conteggio-selezionati"
                           role="status"
                           aria-live="polite"></p>
                        <p class="text-success mt-2 mb-0"
                           id="esito-categoria"
                           role="status"
                           aria-live="polite"></p>
                    </div>
                </div>
                <div class="table-responsive">
                    <table class="table table-striped table-hover">
                        <thead>
                            <tr>
                                <th>Importa</th>
                                <th>Riga</th>
                                <th>Data</th>
```

```
                                    <th>IP</th>
                                    <th>Gravità</th>
                                    <th>Descrizione</th>
                                    <th>Categoria</th>
                                </tr>
                            </thead>
                            <tbody>
                                {% for evento in eventi %}
                                    <tr>
                                        {% set selezionato = request.method != 'POST' or request.form.get('azione') != 'c
onferma' or evento.riga|string in request.form.getlist('selezionati') %}
                                        <td>
                                            <input class="form-check-input selezione-evento"
                                                   type="checkbox"
                                                   name="selezionati"
                                                   value="{{ evento.riga }}"
                                                   aria-label="Importa riga {{ evento.riga }}"
                                                   {% if selezionato %}checked{% endif %}>
                                        </td>
                                        <td>{{ evento.riga }}</td>
                                        <td>{{ evento.data_evento }}</td>
                                        <td>{{ evento.indirizzo_ip }}</td>
                                        <td>{{ evento.gravita }}</td>
                                        <td>{{ evento.descrizione }}</td>
                                        <td>
                                            <select class="form-select"
                                                    name="categoria_{{ evento.riga }}"
                                                    aria-label="Categoria della riga {{ evento.riga }}"
                                                    {% if selezionato %}required{% else %}disabled{% endif %}>
                                                <option value="">Seleziona una categoria</option>
                                                {% for categoria in categorie %}
                                                    <option value="{{ categoria.id_categoria }}" {% if request.form.get('
categoria_' ~ evento.riga) == categoria.id_categoria|string %}selected{% endif %}>
                                                        {{ categoria.nome }}
                                                    </option>
                                                {% endfor %}
                                            </select>
                                        </td>
                                    </tr>
                                {% endfor %}
                            </tbody>
                        </table>
                    </div>
                    <button class="btn btn-applicaFiltri"
                            type="submit"
                            {% if not categorie %}disabled{% endif %}>Importa eventi selezionati</button>
                    <a class="btn btn-secondary"
                       href="{{ url_for('importaLog') }}">Annulla anteprima</a>
                </form>
            {% endif %}
        </main>
        {% include "footer.html" %}
        <script src="{{ url_for('static', filename='js/importa_log.js') }}"></script>
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
    </body>
</html>
```

templates/importazioni.html

54 righe

```
{% extends "base_account.html" %}
{% block titolo %}Storico log{% endblock %}
{% block contenuto %}
    <div class="d-flex flex-wrap justify-content-between align-items-center gap-3 mb-4">
        <div>
            <h1>Storico dei log</h1>
            <p class="text-secondary mb-0">{{ 'Tutte le importazioni degli utenti.' if utente_corrente.ruolo == 'admin' e
lse 'I file che hai importato, sempre a portata di mano.' }}</p>
        </div>
        <a class="btn btn-applicaFiltri" href="{{ url_for('importaLog') }}">Importa un log</a>
    </div>
    <div class="card info-card">
        <div class="card-body">
            <div class="table-responsive">
                <table class="table align-middle">
```

```
                    <thead>
                        <tr>
                            <th>File</th>
                            <th>Utente</th>
                            <th>Data</th>
                            <th>Eventi importati</th>
                            <th>Righe errate</th>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for i in importazioni %}
                            <tr>
                                <td>
                                    {{ i.nome_file }}
                                    <br>
                                    <small class="text-secondary">{{ i.dimensione }} byte</small>
                                </td>
                                <td>{{ i.username }}</td>
                                <td>{{ i.data_importazione }}</td>
                                <td>{{ i.eventi_importati }}</td>
                                <td>{{ i.righe_errate }}</td>
                                <td>
                                    <a class="btn btn-azioni btn-sm"
                                       href="{{ url_for('dettagliImportazione', id_importazione=i.id_importazione) }}">Ap
ri</a>
                                </td>
                            </tr>
                        {% else %}
                            <tr>
                                <td colspan="6" class="text-center py-4">
                                    Nessun file importato. Inizia caricando il tuo primo log.
                                </td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>
{% endblock %}
```

templates/info.html

106 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Info e Contatti - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.min.css">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <section class="text-center mb-5 info-introduzione mx-auto">
                <h1 class="display-5 fw-bold">Info e Contatti</h1>
                <p class="lead text-secondary mb-0">Il progetto, le tecnologie e la persona dietro CS Monitor.</p>
            </section>
            <div class="row g-4">
                <div class="col-lg-7">
                    <section class="card info-card mb-4" aria-labelledby="titolo-progetto">
                        <div class="card-body p-4 p-md-5">
                            <div class="d-flex align-items-center gap-3 mb-4">
                                <span class="info-icona"><i class="bi bi-window"
   aria-hidden="true"></i></span>
                                <h2 id="titolo-progetto" class="h4 mb-0 fw-bold">Il progetto</h2>
                            </div>
                            <p class="text-secondary info-testo">CS Monitor è una web application sviluppata per la gesti
```

```
one degli eventi di sicurezza informatica. Un unico spazio per organizzare le segnalazioni e seguirne l'analisi.</p>
                            <ul class="list-unstyled mb-4 info-funzionalita">
                                <li>
                                    <i class="bi bi-check-circle"
                                       aria-hidden="true"></i> Registra e gestisci gli eventi di sicurezza
                                </li>
                                <li>
                                    <i class="bi bi-check-circle"
                                       aria-hidden="true"></i> Cerca, filtra e consulta le statistiche
                                </li>
                                <li>
                                    <i class="bi bi-check-circle"
                                       aria-hidden="true"></i> Importa i log ed esporta gli eventi in CSV
                                </li>
                            </ul>
                        </div>
                    </section>
                    <section class="card info-card" aria-labelledby="titolo-tecnologie">
                        <div class="card-body p-4 p-md-5">
                            <div class="d-flex align-items-center gap-3 mb-4">
                                <span class="info-icona"><i class="bi bi-code-slash"
   aria-hidden="true"></i></span>
                                <h2 id="titolo-tecnologie"
                                    class="h4 mb-0 fw-bold">Tecnologie utilizzate</h2>
                            </div>
                            <p class="text-secondary mb-3">Dalla logica applicativa al database, fino all'interfaccia web
.</p>
                            <ul class="list-unstyled d-flex flex-wrap gap-2 mb-0">
                                {% for tecnologia in ['Python', 'Flask', 'MySQL', 'HTML', 'CSS', 'JavaScript', 'Bootstrap
', 'Jinja', 'Git'] %}
                                    <li class="info-tecnologia">{{ tecnologia }}</li>
                                {% endfor %}
                            </ul>
                        </div>
                    </section>
                </div>
                <div class="col-lg-5">
                    <section class="card info-card info-profilo"
                             aria-labelledby="titolo-sviluppatore">
                        <div class="card-body p-4 p-md-5">
                            <img class="info-avatar mb-4"
                                 src="{{ url_for('static', filename='img/profilo.png') }}"
                                 alt="Foto di Marco Filannino"
                                 width="76"
                                 height="76">
                            <h2 id="titolo-sviluppatore"
                                class="h6 text-secondary mb-2">Lo sviluppatore</h2>
                            <h3 class="h3 fw-bold mb-3">Marco Filannino</h3>
                            <p class="text-secondary info-testo">Studente di Ingegneria Informatica e dell'Automazione pr
esso il Politecnico di Bari.</p>
                            <div class="info-contatti mt-4 pt-4">
                                <h4 class="h6 fw-bold mb-3">Contatti professionali</h4>
                                <div class="d-grid gap-2">
                                    <a href="https://github.com/MarcoFilannino2004"
                                       target="_blank"
                                       rel="noopener noreferrer"
                                       class="info-contatto">
                                        <i class="bi bi-github" aria-hidden="true"></i><span>GitHub</span><i class="bi bi
-box-arrow-up-right ms-auto" aria-hidden="true"></i>
                                    </a>
                                    <a href="https://www.linkedin.com/in/marco-filannino/"
                                       target="_blank"
                                       rel="noopener noreferrer"
                                       class="info-contatto">
                                        <i class="bi bi-linkedin" aria-hidden="true"></i><span>LinkedIn</span><i class="b
i bi-box-arrow-up-right ms-auto" aria-hidden="true"></i>
                                    </a>
                                    <a href="mailto:marcofilannino04@gmail.com"
                                       class="info-contatto">
                                        <i class="bi bi-envelope" aria-hidden="true"></i><span>Email</span><i class="bi b
i-arrow-right ms-auto" aria-hidden="true"></i>
                                    </a>
                                </div>
                            </div>
```

```
                    </div>
                </section>
            </div>
        </div>
    </main>
    {% include "footer.html" %}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

templates/login.html

32 righe

```
{% extends "base_account.html" %}
{% block titolo %}Accedi{% endblock %}
{% block contenuto %}
    <div class="account-card card info-card mx-auto">
        <div class="card-body p-4 p-md-5">
            <span class="info-tecnologia d-inline-block mb-3">CS Monitor</span>
            <h1 class="h2">Bentornato</h1>
            <p class="text-secondary">Accedi per gestire i tuoi eventi e ritrovare i log importati.</p>
            <form method="post">
                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                <label class="form-label" for="identificativo">Nome utente o email</label>
                <input class="form-control mb-3"
                       id="identificativo"
                       name="identificativo"
                       autocomplete="username"
                       maxlength="254"
                       value="{{ request.form.get('identificativo', '') }}"
                       required>
                <label class="form-label" for="password">Password</label>
                <input class="form-control mb-4"
                       id="password"
                       name="password"
                       type="password"
                       autocomplete="current-password"
                       maxlength="128"
                       required>
                <button class="btn btn-applicaFiltri w-100" type="submit">Accedi</button>
            </form>
            <p class="mt-4 mb-0">Non hai un account? <a href="{{ url_for('account.registrazione') }}">Registrati</a></p>
        </div>
    </div>
{% endblock %}
```

templates/modifica_evento.html

76 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Modifica evento - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <h1 class="mb-4">Modifica evento ID: {{ evento.id_evento }}</h1>
            {% if errore %}
                <div class="alert alert-danger">{{ errore }}</div>
            {% endif %}
            <form action="{{ url_for('modificaEventoPagina', id_evento=evento.id_evento) }}"
                  method="post">
                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                <div class="mb-3">
                    <label for="id_categoria" class="form-label">Categoria</label>
                    <select class="form-select" id="id_categoria" name="id_categoria" required>
                        {% for categoria in categorie %}
                            <option value="{{ categoria.id_categoria }}" {% if categoria.id_categoria|string == dati.get(
'id_categoria', '')|string %}selected{% endif %}>
```

```
                            {{ categoria.nome }}
                        </option>
                    {% endfor %}
                </select>
            </div>
            <div class="mb-3">
                <label for="indirizzo_ip" class="form-label">Indirizzo IP</label>
                <input type="text"
                       class="form-control"
                       id="indirizzo_ip"
                       name="indirizzo_ip"
                       value="{{ dati.get('indirizzo_ip', '') }}"
                       required>
            </div>
            <div class="mb-3">
                <label for="descrizione" class="form-label">Descrizione</label>
                <textarea class="form-control"
                          id="descrizione"
                          name="descrizione"
                          rows="4"
                          required>{{ dati.get('descrizione', '') }}</textarea>
            </div>
            <div class="mb-3">
                <label for="gravita" class="form-label">Gravità</label>
                <select class="form-select" id="gravita" name="gravita" required>
                    {% for gravita in ["Bassa", "Media", "Alta", "Critica"] %}
                        <option value="{{ gravita }}" {% if gravita == dati.get('gravita') %}selected{% endif %}>
                            {{ gravita }}
                        </option>
                    {% endfor %}
                </select>
            </div>
            <div class="mb-3">
                <label for="stato" class="form-label">Stato</label>
                <select class="form-select" id="stato" name="stato" required>
                    {% for stato in ["Nuovo", "In analisi", "Risolto"] %}
                        <option value="{{ stato }}" {% if stato == dati.get('stato') %}selected{% endif %}>
                            {{ stato }}
                        </option>
                    {% endfor %}
                </select>
            </div>
            <button type="submit" class="btn btn-applicaFiltri">Salva modifiche</button>
            <a href="{{ url_for('dettagliEvento', id_evento=evento.id_evento) }}"
               class="btn btn-secondary">Annulla</a>
        </form>
    </main>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

templates/navbar.html

61 righe

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="{{ url_for('home') }}">
            <img src="{{ url_for('static', filename='img/logo.png') }}"
                 alt="Logo CS Monitor"
                 style="height: 45px;
                        width: auto">
        </a>
        <button class="navbar-toggler"
                type="button"
                data-bs-toggle="collapse"
                data-bs-target="#navbarMenu"
                aria-controls="navbarMenu"
                aria-expanded="false"
                aria-label="Apri menu">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarMenu">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('home') }}">Home</a>
```

```
                </li>
                {% if utente_corrente %}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ url_for('dashboard') }}">Dashboard</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ url_for('nuovoEvento') }}">Nuovo evento</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ url_for('importaLog') }}">Importa log</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link"
                           href="{{ url_for('storicoImportazioni') }}">Storico log</a>
                    </li>
                {% endif %}
                <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('info') }}">Info e contatti</a>
                </li>
            </ul>
            <div class="d-flex flex-wrap align-items-center gap-2">
                {% if utente_corrente %}
                    {% if utente_corrente.ruolo == "admin" %}
                        <a class="nav-link text-nowrap"
                           href="{{ url_for('account.amministrazione') }}">Gestione account</a>
                    {% endif %}
                    <a class="nav-link text-nowrap"
                       href="{{ url_for('account.profilo') }}">{{ utente_corrente.username }}</a>
                    <form method="post" action="{{ url_for('account.logout') }}">
                        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                        <button class="btn btn-secondary btn-sm" type="submit">Esci</button>
                    </form>
                {% else %}
                    <a class="nav-link text-nowrap" href="{{ url_for('account.login') }}">Accedi</a><a class="btn btn-app
licaFiltri btn-sm"
   href="{{ url_for('account.registrazione') }}">Registrati</a>
                {% endif %}
            </div>
        </div>
    </div>
</nav>
```

templates/nuovo_evento.html

69 righe

```
<!DOCTYPE html>
<html lang="it">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Nuovo evento - CS Monitor</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
              rel="stylesheet">
        <link rel="stylesheet"
              href="{{ url_for('static', filename='css/stile.css') }}">
    </head>
    <body class="d-flex flex-column min-vh-100">
        {% include "navbar.html" %}
        <main class="container py-5 flex-grow-1">
            <h1 class="mb-4">Registra nuovo evento</h1>
            {% if errore %}
                <div class="alert alert-danger">{{ errore }}</div>
            {% endif %}
            <form action="{{ url_for('nuovoEvento') }}" method="post">
                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                <div class="mb-3">
                    <label for="id_categoria" class="form-label">Categoria dell'evento</label>
                    <select class="form-select" id="id_categoria" name="id_categoria" required>
                        <option value="" {% if not dati.get('id_categoria') %}selected{% endif %}
                                disabled>Seleziona una categoria</option>
                        {% for categoria in categorie %}
                            <option value="{{ categoria.id_categoria }}" {% if dati.get('id_categoria') == categoria.id_c
ategoria|string %}selected{% endif %}>
                                {{ categoria.nome }}
                            </option>
```

```
                    {% endfor %}
                </select>
            </div>
            <div class="mb-3">
                <label for="indirizzo_ip" class="form-label">Indirizzo IP</label>
                <input type="text"
                       class="form-control"
                       id="indirizzo_ip"
                       name="indirizzo_ip"
                       placeholder="192.168.1.10"
                       value="{{ dati.get('indirizzo_ip', '') }}"
                       required>
            </div>
            <div class="mb-3">
                <label for="descrizione" class="form-label">Descrizione</label>
                <textarea class="form-control"
                          id="descrizione"
                          name="descrizione"
                          rows="4"
                          required>{{ dati.get('descrizione', '') }}</textarea>
            </div>
            <div class="mb-3">
                <label for="gravita" class="form-label">Gravità</label>
                <select class="form-select" id="gravita" name="gravita" required>
                    <option value="" {% if not dati.get('gravita') %}selected{% endif %}
                            disabled>Seleziona la gravità</option>
                    {% for gravita in ['Bassa', 'Media', 'Alta', 'Critica'] %}
                        <option value="{{ gravita }}" {% if dati.get('gravita') == gravita %}selected{% endif %}>
                            {{ gravita }}
                        </option>
                    {% endfor %}
                </select>
            </div>
            <button type="submit" class="btn btn-applicaFiltri">Registra evento</button>
            <a href="{{ url_for('dashboard') }}" class="btn btn-secondary">Annulla</a>
        </form>
    </main>
    {% include "footer.html" %}
</body>
</html>
```

templates/profilo.html

81 righe

```
{% extends "base_account.html" %}
{% block titolo %}Il mio account{% endblock %}
{% block contenuto %}
    <h1>Il mio account</h1>
    <p class="text-secondary">Gestisci i tuoi dati e la password. Ruolo: {{ utente_corrente.ruolo }}.</p>
    {% if utente_corrente.password_da_cambiare %}
        <div class="alert alert-warning">
            Stai usando una password temporanea. Cambiala qui sotto per accedere agli eventi e ai log.
        </div>
    {% endif %}
    <div class="row g-4">
        <div class="col-lg-6">
            <div class="card info-card h-100">
                <div class="card-body p-4">
                    <h2 class="h4">Dati personali</h2>
                    <form method="post">
                        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                        <input type="hidden" name="azione" value="profilo">
                        <label class="form-label" for="username">Nome utente</label>
                        <input class="form-control mb-3"
                               id="username"
                               name="username"
                               minlength="3"
                               maxlength="40"
                               value="{{ request.form.get('username', utente_corrente.username) }}"
                               required>
                        <label class="form-label" for="email">Email</label>
                        <input class="form-control mb-4"
                               id="email"
                               name="email"
                               type="email"
```

```
                            maxlength="254"
                            value="{{ request.form.get('email', utente_corrente.email) }}"
                            required>
                        <button class="btn btn-applicaFiltri" type="submit">Salva profilo</button>
                    </form>
                </div>
            </div>
        </div>
        <div class="col-lg-6">
            <div class="card info-card">
                <div class="card-body p-4">
                    <h2 class="h4">Cambia password</h2>
                    <p class="text-secondary">Dopo il cambio dovrai accedere di nuovo su tutti i dispositivi.</p>
                    <form method="post">
                        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                        <input type="hidden" name="azione" value="password">
                        <label class="form-label" for="password_attuale">Password attuale</label>
                        <input class="form-control mb-3"
                            id="password_attuale"
                            name="password_attuale"
                            type="password"
                            autocomplete="current-password"
                            maxlength="128"
                            required>
                        <label class="form-label" for="nuova_password">Nuova password</label>
                        <input class="form-control mb-3"
                            id="nuova_password"
                            name="nuova_password"
                            type="password"
                            autocomplete="new-password"
                            minlength="10"
                            maxlength="128"
                            required>
                        <label class="form-label"
                            for="conferma_password">Conferma nuova password</label>
                        <input class="form-control mb-4"
                            id="conferma_password"
                            name="conferma_password"
                            type="password"
                            autocomplete="new-password"
                            minlength="10"
                            maxlength="128"
                            required>
                        <button class="btn btn-applicaFiltri" type="submit">Cambia password</button>
                    </form>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

templates/registrazione.html

59 righe

```
{% extends "base_account.html" %}
{% block titolo %}Registrati{% endblock %}
{% block contenuto %}
    <div class="account-card card info-card mx-auto">
        <div class="card-body p-4 p-md-5">
            <span class="info-tecnologia d-inline-block mb-3">Il tuo spazio personale</span>
            <h1 class="h2">Crea un account</h1>
            <p class="text-secondary">Tieni insieme i tuoi eventi, i file originali e lo storico delle importazioni.</p>
            <form method="post">
                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
                <label class="form-label" for="username">Nome utente</label>
                <input class="form-control"
                       id="username"
                       name="username"
                       autocomplete="username"
                       minlength="3"
                       maxlength="40"
                       pattern="[A-Za-z0-9_.\-]{3,40}"
                       value="{{ request.form.get('username', '') }}"
                       required>
                <div class="form-text mb-3">
```

```
                    Da 3 a 40 caratteri: lettere, numeri, punti, trattini e underscore.
                </div>
                <label class="form-label" for="email">Email</label>
                <input class="form-control mb-3"
                       id="email"
                       name="email"
                       type="email"
                       autocomplete="email"
                       maxlength="254"
                       value="{{ request.form.get('email', '') }}"
                       required>
                <label class="form-label" for="password">Password</label>
                <input class="form-control"
                       id="password"
                       name="password"
                       type="password"
                       autocomplete="new-password"
                       minlength="10"
                       maxlength="128"
                       required>
                <div class="form-text mb-3">
                    Almeno 10 caratteri. Usa una password diversa da quella degli altri servizi.
                </div>
                <label class="form-label" for="conferma_password">Conferma password</label>
                <input class="form-control mb-4"
                       id="conferma_password"
                       name="conferma_password"
                       type="password"
                       autocomplete="new-password"
                       minlength="10"
                       maxlength="128"
                       required>
                <button class="btn btn-applicaFiltri w-100" type="submit">Registrati</button>
            </form>
            <p class="mt-4 mb-0">Hai già un account? <a href="{{ url_for('account.login') }}">Accedi</a></p>
        </div>
    </div>
{% endblock %}
```